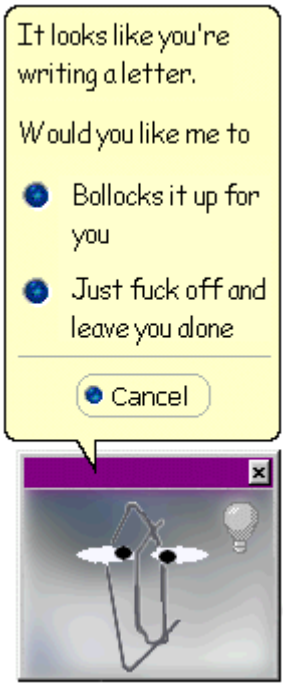
PATN

A revised USER's Guide

25th October 2003



Lee Belbin
Blatant Fabrications Pty Ltd

TABLE OF CONTENTS

READ THIS FIRST

The **USER'S GUIDE** is designed to provide you with an understanding of the *environment* that **PATN** uses. The *Technical Reference* is intended for more specific technical information for each of **PATN**'s algorithms and options.

MODULES

PATN is a collection of over 50 *modules* or separate programs covering a wide range of multivariate data analysis. Each of **PATN**'s interactive modules correspond to a major component of pattern analysis, for example, the module FUSE does hierarchical-agglomerative-clustering. Once a module/option is selected, **PATN** prompts for sub-options (that usually have default values), carries out the requested operation, and usually stores results in one or more files. In some cases, for example FUSE, this file is in ASCII format and is listed to the terminal, may be printed and can used in subsequent analysis. In other cases, for example ASO, the output file is in binary format and cannot be immediately listed to a terminal or printed to a printer without some disaster.

HELP

Extensive on-line help is available by typing a question mark to most **PATN** prompts. The on-line help was designed to be complementary to the information in the Technical Reference. Its purpose is to assist you in understanding the implications of each option and sub-option.

YOUR DATA

Matrices

PATN generally assumes that your data will initially be in the form of a two dimensional matrix in *ASCII format*. This is basically, the same structure as a spreadsheet. If your data was in the form of an EXCEL spreadsheet (a -*.xls* file), you could not list or print this other than by using the EXCEL program. The same is true of **PATN.** Data is always read as ASCII text file into an i**nternal** binary format. This must be done prior to any pattern analysis on a datafile.

PATN assumes that the rows of the matrix are the *objects* and are of primary importance. Objects are the things that you want to know more about. The columns of the matrix are the *attributes* or variables that describe the objects. Attributes are considered secondary to objects.

Parameters

Before **PATN** can begin to analyse your data, it needs to know a number of facts. These basic parameters are:

1. what is the name of the file containing the data
2. how many rows (objects) are in the data and
3. how many columns (attributes).

There are other parameters, but these are less significant at this point. The module PRAM is used to define these parameters and store them in a special file called *PATN.PRM* in binary format. Similarly, the module DATN is used for the input of data to, and the output of data from **PATN**. Each time a **PATN** module is activated, the parameters stored in the file *PATN.PRM* are read. In this way, **PATN** knows about the data that the user is currently interested in.

Object & Attribute Labels

PATN also requires that a set of row and column labels are available to annotate most output. As with the data and parameters, these labels are read by the module LABN either from the terminal or a file in ASCII format and stored in PATN in separate row and column label files in binary format.

A BRIEF EXAMPLE

Following is a simple example of how to use **PATN** to get a basic *classification* from scratch. **PATN** is command driven, so the following example is a list of these commands that results in the production of a dendrogram. This list was taken directly from the *log file, PATN.LOG* (see the chapter on FILES). This special file is used to optionally record (log) all keystrokes while using **PATN**. In this case, extensive logging was activated.

The information after the '!' from the log file are the annotations that **PATN** *automatically* adds. They are used to interpret what the various commands and parameters represent.

First-up, I have used the module PRAM to specify what my data looked like. Then, DATN is used to read the data into **PATN**. Similarly, LABN has been used to read a set of row and column labels into **PATN**. Then, a matrix of associations between all pairs of rows was generated using ASO. Next, the hierarchical classification is performed using FUSE. Finally, a dendrogram is drawn using DEND.

The datafile was entered using a text editor and saved in ASCII format. Word processors such as WORD and WORD-PERFECT can also be used but it must be remembered that these packages will normally save the file with formatting in binary format. Optionally, word-processing packages can save information in ASCII format.

Here is what the input data looks like-

```
ROW00001   1.0   8.3   9.5   0.4   0.1   0.5   7.7   5.8   9.1   7.8
ROW00002   3.3   2.0   2.7   7.9   9.8   1.5   3.4   0.3   2.7   1.7
ROW00003   8.6   2.1   9.2   2.4   9.3   9.4   3.1   6.3   5.1   2.0
ROW00004   9.1   3.3   9.8   9.5   9.3   0.5   9.2   8.9   3.6   1.1
ROW00005   7.0   5.8   2.4   1.8  10.0   3.1   5.3   7.1   6.0   5.0
ROW00006   3.8   6.0   3.3   7.2   1.3   5.3   0.6   5.7   1.7   6.7
ROW00007   9.4   3.4   6.2   4.6   1.0   1.4   5.5   1.0   1.5   4.3
ROW00008   2.3   2.3   5.7   0.8   7.3   5.8   4.0   2.9   8.9   1.8
ROW00009   4.0   6.4   8.9   9.8   5.1   7.1   6.9   2.1   3.1   4.2
ROW00010   7.9   5.4   2.8   3.6   3.0   6.4   1.4   4.5   7.2   0.5
ROW00011   0.2   4.4   6.4   4.6   6.7   0.8   7.0   0.5   7.8   7.4
ROW00012   9.3   6.7   3.4   1.5   7.6   9.8   3.3   4.2   0.9   1.2
ROW00013   2.8   7.9   0.3   4.5   7.9   7.2   3.4   7.0   1.6   0.3
ROW00014   9.0   6.6   8.9   2.2   8.8   4.7   3.0   2.9   8.2   6.3
ROW00015   8.9   5.2   6.2   9.3   9.0   2.2   7.7   6.7   5.0   7.1
ROW00016   4.6   7.2   8.5   0.0   1.4   3.3   2.6   7.3   0.7   9.4
ROW00017   6.1   5.5   0.8   1.3   2.9   1.6   1.1   2.5   7.2   3.1
ROW00018   5.2   3.3   8.8   3.4   2.8   1.1   9.9   9.3   0.3   7.7
ROW00019   8.8   6.5   4.2   5.2   5.7   2.4   5.7   6.4   8.0   3.1
ROW00020   7.3   0.3   8.6   8.0   8.9   4.1   6.3   1.6   5.3   8.9
```

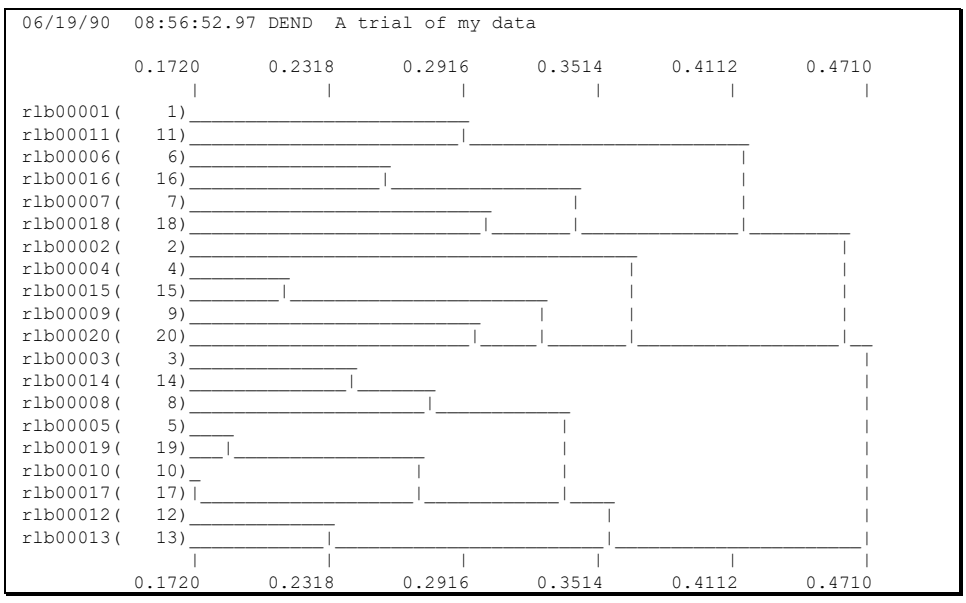
A separate file containing column labels was created using a word-processor. Do **not** however save the file from the word processor in binary format. The ASCII file should look like this-

```
COL00001
COL00002
COL00003
COL00004
COL00005
COL00006
COL00007
COL00008
COL00009
COL00010
```

Default values have been extensively used so there are really very few keystrokes needed for this example. I have not included a listing of any of the intermediate output from **PATN**, only the final dendrogram.

```
PRAM           ! 06/19/90  ! 08:49:52.57 ! RANDOM DATA SET FROM RAND OPTION
          2                              ! PARAMETERS
test.dat                                 ! New data file name
         11                              ! PARAMETERS
PRAM           ! 06/19/90  ! 08:50:12.35 ! RANDOM DATA SET FROM RAND OPTION
          0                              ! PARAMETERS
A trial classification of my data        ! New title
test.dat                                 ! New data file name
         20                              ! Number of rows of data
         10                              ! Number of columns of data
          0                              ! Number of row groups
          0                              ! Number of column groups
  -9999.                                 ! Missing data value
         11                              ! PARAMETERS
DATN           ! 06/19/90  ! 08:54:47.08 ! A trial of my data
          1                              ! DATA I/O OPTION
                                         ! INPUT FILE NAME
(8x,10f6.1)                              ! INPUT DATA FORMAT
                                         ! OUTPUT FILE NAME
LABN           ! 06/19/90  ! 08:55:08.18 ! A trial of my data
          7                              ! LABEL OPTION
          3                              ! COLUMN LABEL OPTION
test.col                                 ! INPUT FILE NAME
(a)                                      ! INPUT DATA FORMAT
ASO            ! 06/19/90  ! 08:56:18.97 ! A trial of my data
          5                              ! ASSOCIATION MEASURE OPTION
FUSE           ! 06/19/90  ! 08:56:28.31 ! A trial of my data
          5                              ! FUSION STRATEGY
          0                              ! ORDER OF OUTPUT ASSOC MATRIX
N                                        ! USE ADJACENCY CONSTRAINT Y/N
 -0.1000                                 ! BETA VALUE FOR FLEXIBLE UPGMA
                                         ! NEXT PAGE OF FILE --> TERMINAL
N                                        ! PRINT FILE ?
DEND           ! 06/19/90  ! 08:56:52.97 ! A trial of my data
         20                              ! NO OF GROUPS TO BE PRINTED
          1                              ! 1=80 COL__2=132 COL__3=80D+50L
                                         ! NEXT PAGE OF FILE --> TERMINAL
N                                        ! PRINT FILE ?
N                                        ! CALCULATE & STORE ULTRAMETRICS
```

And this is the result-



THIS DOCUMENT

IT'S PURPOSE

This *Users Guide* assumes no background, other than possibly a high school education. The aim of this document is to outline the environment that **PATN** uses. Once this is understood, most of the complexities of the package fall into place. By this I mean that I have designed **PATN** to be as consistent as possible across most of its operations. For those in need, I would recommend that at least one of the introductory texts listed in the following section should be reviewed to supply some background in pattern analysis.

PATN is not what I would call a high-level package. It is not the type of program that you can simply start it and say, "give me an analysis" and expect a intelligent result. There are precious few such programs that give a reasoned result. At least not without a good degree of interaction with the analyst. Using **PATN** is like working with lego, you have to build up the result step-by-step.

Some computing experience is an advantage for the efficient use of **PATN**. The main reason for this is that **PATN** uses the *file structure* supplied by the operating system (the program that allows the computer to be more easily used). It also requires a little knowledge of FORTRAN formatting conventions to get data into and out of the package. A separate chapter on FORTRAN formatting is included so don't panic (it is fairly simple and constant).

The **PATN** environment is covered in the first section of this document. A summary can be found in the chapter called **OVERVIEW** and should be used as a primer to identify the limits of your knowledge. A grasp of this information will be required to use **PATN** effectively. Don't use **PATN** as a *black box*. It was designed specifically not to be used as such. Like most other packages, **PATN** has a large number of pathways, options, files and associated data structures.

DOCUMENTATION

PATN documentation is divided into two manuals-

1. a USERS GUIDE and
2. a TECHNICAL REFERENCE

The Users Guide

This manual is intended to outline the general environment and structure of PATN. The organisation of this document is designed to provide answers to the questions:

1. WHAT is **PATN** ?
2. WHY use **PATN** ?
3. HOW **PATN** is used ?

The Users Guide is intended to provide a first point of contact and to be used in a sequential fashion. The chapters on *files* and *formats* may be scanned or skipped as required. Use the table of contents to get an overview of the structure so that subsequent details can be understood in context. A detailed index and glossary are provided. Use them.

Technical Reference

Provides details of each **PATN** module and is organised by *function*; the layout follows.

PATN STRUCTURE

Pre-Processing

Data specification (PRAM)
Data input and output (DATN)
Label input and output (LABN)
Association measure input and output (ASON)
Masking rows and columns of data (MASK)
Sampling and sorting data (SAMP)
Data generation by statistical variates (RAND)
Histograms and univariate statistics (HIST)
Summaries of presence/absence data (SCAN)
Bi-variate scatter plots (SCAT)
Data transformations (TRND)
Association measure transformations (TRNA)

Analysis

Generating measures of association (ASO, GASO)

Hierarchical Clustering

Polythetic Agglomerative (FUSE)
Polythetic Divisive (PDIV, TWIN)
Monothetic Divisive (MDIV)
Printing dendrograms (DEND)
Defining or manipulating a set of groups (GDEF)

Non-Hierarchical Clustering

Multi-step allocation (ALOC)

Ordination methods

Multidimensional scaling (SSH)
Principal Co-ordinates (TRNA+PCA)
Principal Components (PCA)
Reciprocal Averaging/Detrended Correspondence Anal. (DCOR)
1d-Seriation using parsimony (SERE)
Orthogonal rotation methods (PCR)

Network Methods

Nearest neighbour lists (NNB)
Minimum spanning trees (MST)
Bond analysis (BOND)

Others

Minimal-set reserve selection (MSET)
Maximal differences (MAXD)

Post processing and Evaluation

Merging of results and data (MERG)
Relating variables to clusters (GSTA)
Two-way tables (TWAY)
Displaying dendrograms (DEND)
Comparing classifications (RIND, TRNA)
Relating variables to ordinations (PCC SCAT)
Comparing ordinations via Procrustes rotation (PROC)
Displaying groups on a map (COLR)
Displaying groups, ordinations and MST in 3d (TSPN+SPIN)
Monte-Carlo testing of groups (ASIM)
Monte-Carlo testing of attributes in ordination (MCAO)
Monte-Carlo testing of ordination dimensions (MCSS)

Included in each module of the Technical Reference is-

an OUTLINE of the algorithm and methods,
a set of REFERENCES,
the major OPTIONS (top level) available and,
INPUT and OUTPUT.

WHY USE PATN ?

"All the real knowledge which we possess depends on the methods by which we distinguish the similar from the dissimilar. The greater number of natural distinctions this method comprehends, the clearer becomes our idea of things. The more numerous the objects which employ our attention, the more difficult it becomes to form such a method and the more necessary."

Linnaeus: Genera Plantarum (1737).

DATA EXPLORATION

When confronting new data, there are often limited notions of the nature of, and reasons for the variation in data. **PATN** has primarily been designed to address this.

When dealing with volumes of data, preconceptions and misconceptions may be responsible for restraining progress or perceiving a new paradigm. **PATN** procedures, are objective in the sense of bringing no memory to bear on the problem and being confined to the boundaries of the set of data under examination. **PATN** often highlights new features or relationships that were not seen by the original investigator. While the human brain has an uncanny ability to progress through a problem by what appears to be a series of inferential leaps (shortcuts), **PATN** algorithms are heuristic, iterating on simple rules to provide a solution. Pattern analysis algorithms are notorious for their requirement of computing time and memory, even for small problems.

Pattern analysis algorithms, while designed to exhibit patterns in data, may impose patterns of their own. This should not present serious problems to those who understand the methods but may be hazardous for the uninitiated. Each method has its advantages and disadvantages. Each presents a summary of the data that should provide insights into the data or possibly, limitations of the investigator. A single method will rarely provide all the information. Selecting an appropriate classification, ordination and network technique, based on a robust measure of association should provide a useful set of overlapping perspectives.

Ordination

Ordination methods produce a summary by reducing the number of significant variables. Such techniques attempt to condense most of the information contained in all attributes into 2 or 3 new attributes with minimal information loss. If this can be achieved, the *objects* may be conveniently displayed on a single bivariate plot. In this plot, the distance between objects represents the degree of similarity or difference between the objects as measured by the full set of attributes.

Objects that are close in this reduced space are those that are similar. Conversely, those that are separated by large distances are dissimilar in terms of their attributes. A very powerful product of this form of display is that overall *trends* or *gradients* may be more clearly perceived. The original application of the Principal Components method of ordination was to extract trends. No *classification* of objects is performed. The methods will provide an indication if true or *natural* clusters exist. By comparing the reduced space with intrinsic (used in the ordination) or extrinsic attributes, any evident trends may be named (typology inferred) and processes identified.

Clustering

Cluster analysis, as its name suggests, produces clusters or groups of objects. Clustering reduces a set of objects to groups of objects. Groups may be generally defined as containing objects that have a greater degree of similarity to members of their group than to members of other groups. Clusters are defined regardless if *true* or *natural* clusters exist. If they don't, the groups while being readily *typed* (have recognisable qualities) tend to merge into one another with no clearly defined boundaries. The benefit of clustering is that identification of the clusters, natural or not, presents direct evidence for the variation in the data.

There is less direct evidence about the underlying driving forces. For example, examination of clusters containing sites where biological specimens have been collected may, while suggesting an altitude gradient, also suggest that individual species are entering and exiting along this altitude gradient. This may suggest that no well defined *communities* exist. This is useful information. In some respects, classification provides a more digestible summary than ordination. People have problems trying to communicate the *continua* that ordination presents. Breaking a continua into discrete units will often lend itself to simplified communication.

Networks

The term *network* in **PATN**, refers to techniques that primarily form linkages between objects. No clustering is involved. Unlike ordination and classification, these methods require minimal transformation of basic association values. Unlike ordination and clustering, these methods focus more on the *local* neighbourhood of each object. These features make network techniques a useful adjunct to alternate methods, resulting in a different perspective that is readily superimposed, either on a classification or an ordination.

DATA REDUCTION

Reducing the number of ATTRIBUTES.

Surveys, being costly aspects of research and development, need to be efficient. If the volume of data to be measured can be reduced, considerable cost savings will result. Initial or pilot surveys usually attempt to cover the majority of variation by using a wide range of attributes. *Pattern analysis* techniques can be useful in increasing the efficiency of subsequent surveys. Results are often presented that imply that a considerable proportion of the variation can be described by a small sub-set of the original attributes. If this occurs, some or many of the attributes may be discarded. In some circumstances attributes may be able to be combined into more powerful attributes.

Reducing the number of OBJECTS.

Some datasets may be too large to analyse directly. Census data is one example where it may be impossible to analyse trends between individuals. The significance of trends in the data only becomes evident at a higher level of aggregation of the data. Clustering methods produce groups. Once defined, centroids can be used to substitute for the individuals in the groups. For example, clustering using the commands:

ASO
FUSE
DEND
GDEF
GSTA

or

ALOC

could be used to produce a set of groups. Because ordination methods are the most computationally expensive algorithms in PATN, group-centroids may be used to replace objects. In fact, if there are more than around 500 objects, using centroids or some other form of sampling may provide the only method for ordination. Needless to say, ordinating as many as 500 objects may be asking for a very cluttered display that may be difficult to interpret.

HYPOTHESIS GENERATION

PATN is an ideal tool for generating ideas about how processes are determining data variation. Pattern analysis methods are *hypothesis generating* in contrast to the more formal statistical approach of *hypothesis testing*.

As an example, pattern analysis of biological data from the Nullarbor Plain of southern Australia pointed out an anomaly. Examination of photographs of sites that were clustered together, showed one with radically different vegetation structure. The odd site appeared to have nothing in common with the rest. Returning to the analysis, it appeared that the reason for clustering in the odd site was because of similar bird populations. Why should they have had a similar bird population when the vegetation was so different? Further examination revealed that one of sites was disturbed by a fire some years prior to sampling. While no external evidence remained to the biologists, it appeared that the birds had a 'memory'; perceiving the site as it would become, not as it was. This opens up some ideas.

HYPOTHESIS TESTING

Pattern analysis techniques are not normally used to test hypotheses. In some cases however, simple comparisons or testing is feasible. For example, a previous study, numerical or otherwise, may have defined a set of groups. New data has become available and it is required to allocate this data to these *pre-defined* groups. The ALOC module may be used to assign the new samples to the closest group centroids. In addition, any new sites with different characteristics may be identified. While the distances of the samples to all groups is an indication of reliability, no probabilities can be easily assigned without making additional assumptions about the nature of the data.

AN OVERVIEW OF PATN

ITS FUNCTION

PATN was born in 1981 in CSIRO. It was designed as a workbench for research into methods of pattern analysis that could be useful in analysing vegetation patterns. Since that time **PATN** has developed in response to use by a wide variety of people. One of the main features of **PATN** is its flexibility in data handling; it can accept and manipulate a wide variety of data types and structures. **PATN** provides a wide range of commands for *pre-processing* and *analysis* of any data that can be represented by a two, or in some cases, a three dimensional matrix. Within this document, the *rows* of this matrix are usually the *objects* while the *columns* refer to the *attributes*. Objects can be anything for which attributes are quantifiable. The data matrix below shows one form of data suitable for **PATN**.

```
1.0   8.3   9.5   0.4   0.1   4.3   1.1
0.5   7.7   5.8   9.1   7.8   2.3   2.1
3.3   2.0   2.7   7.9   9.8   1.3   1.2
1.5   3.4   0.3   2.7   1.7   0.5   2.3
8.6   2.1   9.2   2.4   9.3   0.7   3.4
9.4   3.1   6.3   5.1   2.0   9.1   4.5
9.1   3.3   9.8   9.5   9.3   2.4   5.6
0.5   9.2   8.9   3.6   1.1   6.2   6.7
7.0   5.8   2.4   1.8  10.0   0.9   7.8
3.1   5.3   7.1   6.0   5.0   7.7   8.9
```

OBJECTS AND ATTRIBUTES

The following are examples of objects to be analysed and their corresponding attributes:

companies	by	turnover, employees, shares traded..
regions	by	flora, fauna, topography, geology
rocks	by	chemical and physical attributes
products	by	user responses in categories
animal	by	presence/absence of skeleton features..
wines	by	chemistry
markets	by	income, preferences
TV programs	by	quality, advertising costs
population	by	ethnicity, income ...
landform	by	slope, slope length, topo-sequence..
patients	by	responses to operation
images	by	spectral classes
people	by	educational subject scores

A second type of data *structure* that can be accepted by **PATN** is the matrix of associations. This is often a symmetric form of matrix where the entries represent relationships between objects. This type of *raw* data is common in sociological and psychological studies. For example, a group of people may be asked to rank preferences for different products. The result is a matrix of similarities or dissimilarities between the various products used for testing. For example, the *association*, measured as a dissimilarity (difference on a scale of 0-1) between objects one and two in the example below is 0.4223.

```
0.4223
0.7244  0.2615
0.6401  0.5753  0.5354
0.4971  0.4048  0.3124  0.6117
0.5177  0.4190  0.4147  0.4761  0.2557
0.5257  0.2629  0.2444  0.6245  0.1295  0.2347
0.1455  0.3100  0.5959  0.5137  0.4536  0.4065  0.4588
0.5810  0.3679  0.2676  0.5246  0.2287  0.3837  0.3000  0.5388
0.3930  0.2125  0.2797  0.4681  0.3219  0.2519  0.2741  0.2932  0.3421
```

The common theme in Pattern Analysis is the exploratory analysis of the structure of data. The aim is to present data in a form that facilitates a more complete understanding of the information it contains and the processes that generated it.

PRE-PROCESSING

PATN provides a wide-range of techniques that could be referred to as *Pre-processing*. Such methods apply to data entry, data generation, manipulation and summary. These procedures are best considered as preparatory to the core of pattern analysis and often take considerably more time than the analysis itself. It should not be difficult to get existing data into **PATN**. Three modules have been designed to provide for the input and output of various data structures-

DATN data matrix input and output
ASON association matrix input and output
LABN row and column label input and output

If a matrix of association values was generated by a program other than **PATN**, ASON would usually be able to read it into an internal **PATN** file.

Files are, by default, assumed to be in the form of a matrix of values, where every value in the matrix is required to be nominated by a *real* or *missing* value. Initially, all data must be able to be read by FORTRAN in standard *ASCII* format, either as

FIXED format or
FREE format.

Sparse Data

In many applications, a data matrix contains a large proportion of zeros that represent *absences* of attributes (eg, species). This type of data in often coded in a form whereby **only** the presences (as 1's) are directly coded. **PATN** can input and output data in this form through DATN.

To make existing files available to **PATN**, all that is required is the nomination of the data file *name*, *number of rows and columns* in the matrix and a value to be used to flag *missing data*. This is done through PRAM. DATN will, depending on data structure, require some type of FORTRAN format. For free format data, an '*' is sufficient to signify that spaces and/or commas separate the data values.

Alternatively, a test dataset can be generated by RAND to get accustomed to **PATN** without worrying about corruption of real data. RAND can also be used to generate data with particular properties that can either be used to expose particular properties of methods or to compare and contrast methods.

PATN can be used to obtain histograms and univariate statistics (means, standard deviation, quartiles, minima, maxima, ranges ...) of either rows or columns of data. Bi-variate scatter plots can be produced with a large combination of plotting, scaling and annotation options available.

PATN provides a variety of options for data manipulation. Reformatting options support two *compressed* methods of coding data where the number of zero values predominate. DATN can also archive or retrieve data using a simple ASCII approach. Sub-setting (masking) and re-ordering of rows or columns of data either extrinsically (direct selection) or intrinsically (based on data values) is supported by MASK. MERG permits the merging of a variety of data (group numbers or order, ordination scores, frequencies or other data files) to the right hand side of **PATN** data files. A variety of sampling and sorting procedures are also provided in SAMP.

Transposition of data (exchange of rows and columns of a matrix) is available for the analysis of attributes. Over a dozen different methods are available for data *transformation* (for example, taking the log of values), *standardisation* such as equalising weights using a variety of methods and *recoding* (for example linear interpolation). A transformation procedure for association matrices includes many of the options available for data files and, in addition, the ability to add or subtract multiple matrices using transformations.

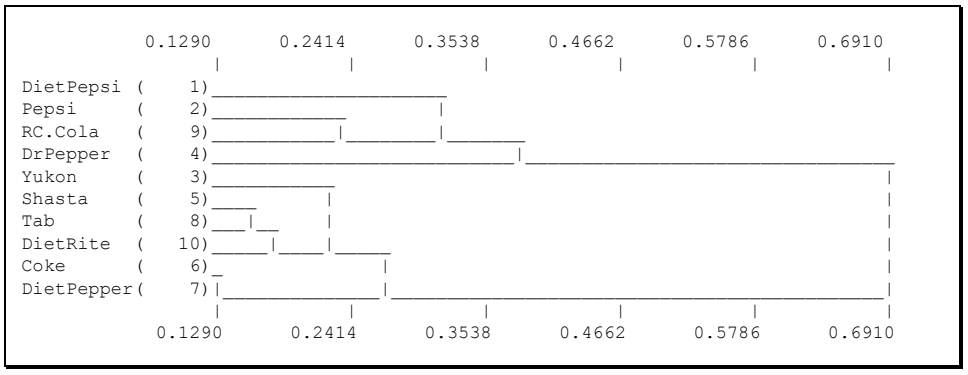
ANALYSIS

Central to Pattern Analysis is the determination of association between pairs of objects in the data. **PATN** provides a wide range of options for this. Attributes that require different and independent measures of association and a complex weighting scheme can also be handled.

PATN provides analysis methods for clustering, ordination and networks. Clustering techniques can themselves be classified as-

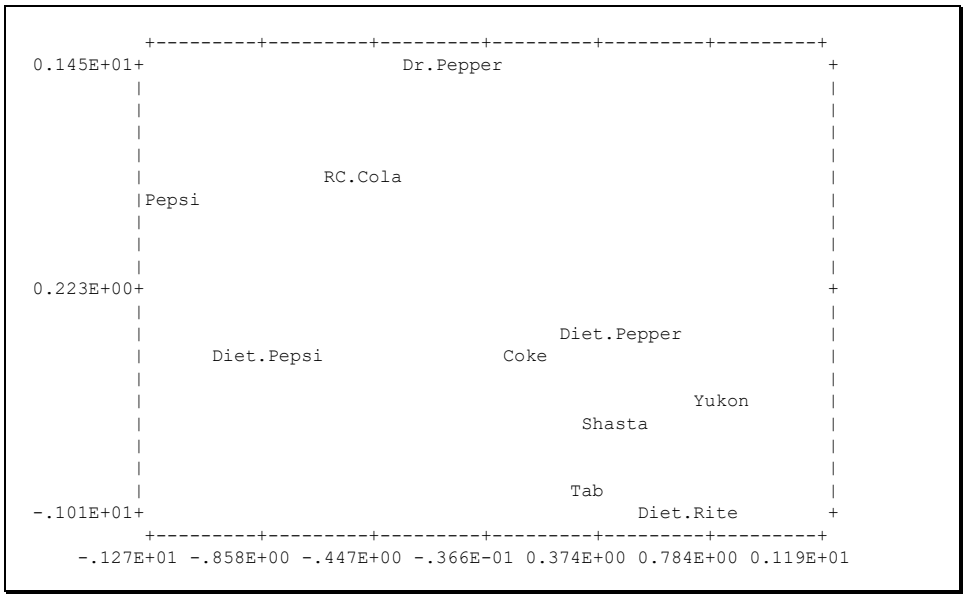
1. hierarchical or non-hierarchical
2. agglomerative (fuse) or divisive (divide)
3. monothetic (one attribute) or polythetic (many)

For example, FUSE is an hierarchical-agglomerative-polythetic clustering method. The tree-like diagram below, called a dendrogram displays the results of clustering using FUSE.



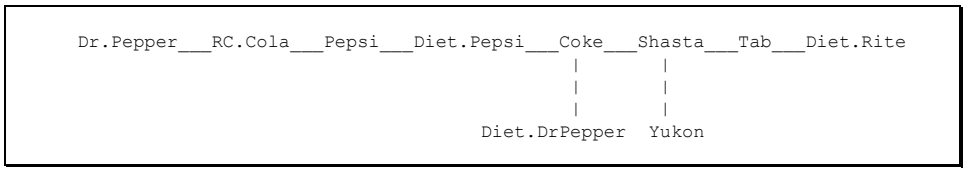
ORDINATION methods are designed to display the objects in a reduced dimensional space with minimal loss of information. The new attributes hopefully account for most of the variation in the data. Ordination methods in **PATN** include the principal axis methods of principal components, principal co-ordinates, reciprocal averaging or correspondence analysis, detrended correspondence analysis and multidimensional scaling.

An example of ordination output is shown below. There are 12 objects displayed (numbered 1 to 12). Each represents one object (row in the data matrix); for example, 1 may represent 'Coco-Cola'. The relationships between the objects is displayed in terms of distance in the diagram. For example Coke and Diet Pepper are close while Diet Rite and Dr Pepper are distant.



NETWORK techniques define a set of connections between objects. Unlike classification and ordination methods, the network algorithms are the result of using only *raw* association values; no averaging or transformations are necessary. The results do not involve any concept of stress or goodness of fit. Methods available include nearest neighbour tables, a concept of 'bond-strength' based on Williams' TWONET algorithm (NNB and BOND) and minimum spanning trees (MST). These techniques have proved to be invaluable as complimentary methods in evaluating ordinations.

An example of a MST would appear as:



POST-PROCESSING/EVALUATION

Subsequent processing can usually enhance the forms of display that are inherent in the usual analysis procedures. Wherever possible, **PATN** provides facilities for graphical displays that are more readily interpreted.

Another aspect of post-processing is the determination of *why* a particular method of analysis produced the results it did. A number of options address this problem. Pattern analysis methods, while often being difficult to implement in an algorithm, are usually simple in concept. Results are rarely difficult to interpret. In some cases however, data may be complex or just so large that normal display methods are inadequate. **PATN** provides a number of ways of expressing results by plotting or tabulation that facilitates data summary and interpretation. **PATN** encourages the overlaying of results from different analysis procedures and intrinsic or extrinsic data.

MODULES

PATN's interaction with the user is by a combination of *menus* and *prompts*. If you use one of the *front-ends* to **PATN**, the structure of the various modules will be apparent. Subsequent screens may provide further options from which to select or prompts for further user input. Default options are provided wherever possible.

PARAMETERS

PATN has two types of PARAMETERS:

1. environmental and
2. command.

Environmental parameters

The *environmental parameters* detail the name and nature of the current dataset and level of logging. While different parameters are used by different commands and options, all *environmental* parameters should be accurate. Once defined, they can be modified at any time, either by the user or by **PATN**. These parameters are maintained in a binary form (unreadable by you) in the file *PATN.PRM*

An example of the *contents* of the file *PATN.PRM*-

```
RANDOM DATA SET FROM RAND OPTION
RANDOM.DAT
     10     5     0     0     0   -9999.
```

These parameters are, in the order as shown above:

1. A *title*
2. the current *data file name*
3. the number of *rows* (*objects*) in the data,
4. the number of *columns* (*attributes*),
5. the number of *row groups*,
6. the number of *column groups*,
7. the level of *logging* currently active and
9. the value to be recognised as *missing* data.

.

These parameters minimise the number of questions **PATN** needs to ask. They also make possible the analysis of a variety of different sets of data on the same disk or directory without any mix-ups.

Command Parameters

Command parameters, are parameters that the user enters in response to **PATN** prompts. These parameters can be alpha-numeric strings, integers, floating-point values or *Yes* or *No*. They determine the action of the module being currently run. Some commands involve requesting many parameters, hopefully most will be default values.

Default values

Throughout **PATN**, *default* values are used to save time, typing, mental effort and errors. These values are those that are supplied by **PATN** when the <*return*> *key* is pressed in response to most **PATN** PROMPTS. These defaults are listed on the same line as the prompt from **PATN**. An example,

How many axes do you want (I,D:2) ? : 3

This states that the default (integer) value is for *two* axes. The value of *three* overwrites the default. The *default* values are provided from research and experience. Considerable work has gone into evaluating certain measures of association, for example the *Bray & Curtis* association measure when using the ASO command. While this is true in the majority of circumstances, **PATN** freely allows choice of many different measures of association. This does not relieve you from an understanding of association measures, rather, it provides direction for further reading.

Defaults are often determined from the *context* of *environmental parameters* to assist you in choosing an appropriate response. For example, GDEF option 1. When requested to define a set of groups from some previous classification, **PATN** will work on the assumption that a *reasonable* number of groups is the *square root* of the number of *objects*. Experience suggests that this is a reasonable starting point.

The implications of some parameters are not obvious, therefore the context and background to all parameters should be carefully examined and understood. This can be done when on DOS by typing a question mark (?), in response to any **PATN** command-parameter prompt, or by examining the *Technical Reference.*

FILES

PATN may create one or more output files for each module run. **No file are deleted** by **PATN** unless you specifically request it. If the file exists and **PATN** is told to create a new version, the old version will be lost (probably forever)!

Filenames

Throughout this document, when the ROOT (the characters to the left of a period) of a file name is not important, a hyphen is used (-). For example, the file output from the ASO command would be referred to in a general sense as:

-.aso

Unformatted or Binary Files

PATN stores the *basic* data files in an *unformatted* form. There are two reasons for this-

1. to speed up input and output of data
2. to by-pass FORTRAN formatting.

Such files are **not** in ASCII format and cannot be edited, TYPEd to the terminal or PRINTed to a standard line-printer. The unformatted files and the **PATN** modules that manipulate them are:

PATN.PRM (the reserved parameter file): PRAM
-.dat (your data file): DATN
-.rlb (the associated row label file): LABN
-.clb (the associated column label file): LABN
-.prm (the saved copy of **PATN**.PRM for *-.dat*): PRAM
-.aso (association measures between pairs of rows in *-.dat*): ASO

The modules PRAM, DATN, LABN and ASON are used to translate between standard ASCII files and their unformatted equivalents. If you are uncertain about the contents of an unformatted file, use the appropriate module to create a formatted (ASCII) equivalent that may be edited and if necessary, read back into **PATN**.

OUTPUT

Files

A FILE is the basic unit of information storage and can be thought of as a collection of records or lines of information that are inter-dependent and related to a theme of the associated command. For example, the ASO command will generate a set of measures of association (proximity, distance, affinity), between each pair of *objects* of data. All values are stored in the file -*.aso* in a form where the position denotes a particular comparison. If there were 3 objects, the file from ASO would contain (in unformatted form) values like this:

```
0.1234
0.2345  0.7654
```

where the first value (0.1234) refers to relationship between objects 1 and 2, the second value (0.2345) to the comparison between 1 and 3 and the third (0.7654) to 2 and 3. The three values are each a measure of *association* between objects in the dataset and the position details which comparison. Generally speaking, **PATN** knows this and you won't have to. It will however pay for you to understand the various data *structures* that **PATN** can produce.

Printing & Typing

PATN makes the distinction between *typing* and *printing* files. If an *ASCII* file results from running any **PATN** module, the first page of this file will be displayed to the terminal (or log file if running in batch mode on a mainframe). ASCII files are files that may be listed, edited or printed. *Unformatted* files will not be displayed, because they contain data in an unprintable form. If an **ASCII** result file contains multiple pages of text, **PATN** will display the file, one page at a time for each <CR> pressed. To abort the listing enter an **S or s** (STOP!) followed by a <CR>. Listing the file like this is effectively the same as TYPING it. Once the complete file is listed, or an **S** has been entered, an option is then provided to print the file to a standard line printer. The file is always stored, so printing, re-naming or deleting may be done at leisure.

MODES OF OPERATION

PATN is designed for *interactive* use. Depending on which implementation you have, **PATN** may also operate in *batch* and *non-interactive* mode using exactly the same set of commands as in interactive mode. *Batch* operation refers to a **PATN** job that is not actively monitored, usually executing in a queue where the user has little or no access once the job has been initiated. **PATN** can generally accept a previously generated log file or some modification of it as input. This enables the user to re-execute failed procedures or analyses of multiple sets of data. *Non-interactive* is where a set of commands for **PATN** have been placed in a file and submitted to **PATN** for execution and with the results echoed to the terminal.

LIMITATIONS

Different options in **PATN** have different requirements and limitations. The most important parameters to the size of a task are usually determined by the data parameters. Generally speaking, virtual memory systems can accommodate whatever the hardware and or operating system can support while standard (640K) MS-DOS systems are limited to around 100,000 numbers. The problem with the standard MS-DOS version is that the program takes up memory that could be used to store data. The extended MS-DOS version (for 80386 and 80486 processors) or UNIX versions are not so limited. Each **PATN** module lists the number of bytes required to process the data with the supplied parameters.

Each **PATN** module requires certain data and parameters to be present. This means that some modules require others to have been previously run. For example, it is unlikely that a *post-processing* command will work if there is no suitable data in files for it to operate on. Similarly, an analysis command cannot operate without the necessary information about where the data is and what is its structure. If you are approaching **PATN** as a novice, this means that you should:

1. Make sure **PATN** knows sufficient information about your data to function (see *environmental parameters*),

2. Be certain that the *data* and *parameter* requirements for each command are met. This is detailed in the Technical Reference for each specific **PATN** module.

AN EXAMPLE.

The following is intended as a simple example of the use of **PATN** and includes:

1. Initiating the *environmental parameters* (PRAM)
2. Reading data into **PATN** using DATN
3. Generating a default set of labels using LABN
4. Generating association (ASO)
5. Performing a Hierarchical cluster analysis (FUSE) and
6. Generating the resulting dendrogram (DEND).

Parameters

Firstly, the specifications of the data are established using the module PRAM. This is used to state the file name and the number of objects and attributes.

```
*PATN< PRAM

Title - Description of analysis status........ A Title
Data File Name (extension assumed -.dat)...... FRED.dat
Number of Rows (Objects) in data matrix.......    10
Number of Columns (Attributes) in data matrix.     5
Number of Row GROUPS.........................     0
Number of Column GROUPS......................     0
Missing Value.............................. -9999.

Logging (0=OFF_1=LIMITED_2=FULL).............     0
```

Data Input

Data must be read into **PATN** before any other operations can be performed. The data here is assumed to be in standard ASCII format, with values taking 6 columns each with 2 decimal places. In **PATN**, the module DATN is used for input and output of data in various forms. The procedure is:

```
-----DATA INPUT AND OUTPUT OPTIONS:

    1 = ASCII --> PATN
    2 = PATN  --> ASCII
    3 = DECORANA 0/1 --> PATN
    4 = DECORANA 0/N --> PATN
    5 = RECODE NOMINAL OR RATIO ATTRIBUTES TO BINARY
    6 = ARCHIVE DATA FILE
    7 = RETRIEVE ARCHIVE FILE
    8 = TRANSPOSE DATA AND LABEL FILES
    9 = ENTER DATA DIRECTLY
   10 = EDIT DATA  (I,D:1)                          ? : 1

    INPUT FILE NAME (A43,D:
    <FRED.DTA                                   >   ? :

.....CURRENT (DEFAULT) FORMAT IS :
(10F6.2)
-----ENTER INPUT FILE FORTRAN FORMAT:

    OUTPUT FILE NAME (A43,D:
    <FRED.DAT                                   >)  ? :

.....Parameters saved in file : RANDOM.prm

*****WARNING: NO ROW/COLUMN LABELS PRODUCED - USE LABN TO CREATE THEM
```

Labels

A set of row and column labels can then be produced using LABN, the counterpart to DATN. For this example, a default set of names will be generated. The row labels are given the names ROW00001, ROW00002...ROW00010 and the column labels COL00001, COL00002....COL00010. These labels are stored in the files *fred.rlb* and *fred.clb* respectively.

```
.....LABN: CURRENT PARAMETERS ARE    10 ROWS AND     5 COLUMNS

-----ROW LABEL INPUT/OUTPUT OPTIONS:

    1 = AUTO-GENERATE --> PATN
    2 = ENTER/EDIT FROM KEYBOARD --> PATN
    3 = ASCII FILE --> PATN
    4 = PATN --> ASCII FILE
    5 = PATN --> TABULATED FILE
    6 = MATCH TWO SETS OF ROW LABELS
    7 = NONE OF THE ABOVE (I,D:1)                    ? : 1

    BASE NAME FOR LABELS (A3,D:ROW)                  ? :

-----COLUMN LABEL INPUT/OUTPUT OPTIONS:

    1 = AUTO-GENERATE --> PATN
    2 = ENTER/EDIT FROM KEYBOARD --> PATN
    3 = ASCII FILE --> PATN
    4 = PATN --> ASCII FILE
    5 = PATN --> TABULATED FILE
    6 = MATCH TWO SETS OF COLUMN LABELS
    7 = NONE OF THE ABOVE (I,D:1)                    ? : 1
    BASE NAME FOR LABELS (A3,D:COL)                  ? :
```

Generating Associations Between Rows

The relationship or association between the ten objects in the file *fred.dat* can now be quantified. A variety of options are available. For this example, the default measure, called the Bray and Curtis coefficient will suffice (see the Technical Reference for further details)-

```
PATN< ASO

-----ASSOCIATION MEASURES:

    1 = BRAY-CURTIS
    2 = CANBERRA METRIC
    3 = CORRELATION COEFFICIENT
    4 = MINKOWSKI (MANHATTAN) SERIES
    5 = GOWER METRIC
    6 = TWO STEP
    7 = ENTER MULTIPLIERS FOR P/A A-B-C-D (1-SIM)
    8 = C - COEFFICIENT
    9 = KENDAL'S SUM OF MINIMUM (COMPLIMENTED)
   10 = SMITHS DISTANCE
   11 = RELIABILITY MEASURE
   12 = CHORD DISTANCE
   13 = SPEARMANS RANK ORDER
   14 = ORDER COEFFICIENT (P/A)
   15 = PROFILES OR 2D ATTRIBUTES
   16 = CHI-SQUARED DISTANCE
   17 = COSINE (OCHIAI) DISTANCE
   18 = YULE'S COEFFICIENT (P/A)
   19 = KULCZYNSKI COEFFICIENT
   20 = ITERATIVE ATTRIBUTE WEIGHTING (I,D:1)       ? : 1
.....ASO: WORKING
```

Association Output

ASO produces a binary, not an ASCII file. A listing of these values can however be produced by using the module ASON (the counterpart to DATN and LABN for association matrices). Note that the structure of the matrix printed below is symmetric about the diagonal. The reason for this is that the association between object 1 and 2 is the same as the association between object 2 and 1!

The values listed below range from zero, implying that the two objects are identical (zero distance apart) to one, implying that they are completely dissimilar. The diagonal is not always calculated because, as in this example, it is assumed to contain all zeros.

```
----------------------------------------<RANDOM.SYM>
0.3220
0.2793  0.3222
0.2730  0.2457  0.2947
0.3827  0.3014  0.2269  0.2301
0.2418  0.2730  0.2830  0.2233  0.3706
0.4897  0.4696  0.2671  0.3531  0.2795  0.5336
0.2663  0.2723  0.1993  0.2524  0.2897  0.3210  0.3441
0.4321  0.2361  0.4185  0.2611  0.3504  0.2874  0.5877  0.3493
0.3084  0.3484  0.3134  0.2124  0.3574  0.2987  0.3632  0.2346  0.2783

.....Print this file to the PRINTER (Y/N,D:N)      ? :
```

Hierarchical Clustering

The hierarchical clustering strategy called FUSE, is the most common method for performing cluster analysis. Again, there are a range of options and sub-options available but the default strategy and parameters are recommended.

A simplified explanation of FUSE is that it scans the association matrix above to find the smallest value. This value represents the closest pair of objects in the data. These are then FUSEd together and a new association between this new group and all other objects is calculated using averages. The process then repeats itself (iterates) until there is only one group remaining (the right-hand side of the dendrogram).

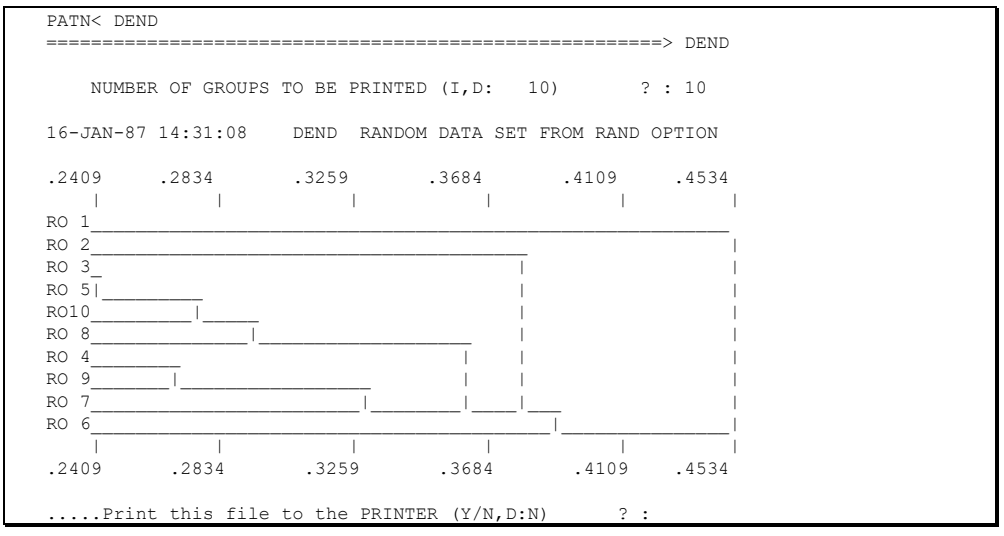
```
PATN< FUSE
================================================> FUSE

-----FUSION STRATEGIES:
    1 = NEAREST NEIGHBOUR
    2 = FURTHEST NEIGHBOUR
    3 = FLEXIBLE WPGMA (SUPPLY: BETA)
    4 = GENERALIZED (SUPPLY:ALPHA, BETA AND GAMMA)
    5 = FLEXIBLE UPGMA (SUPPLY: BETA)
    6 = WPGMA (WEIGHTED GROUP AVERAGE)
    7 = UPGMC (UNWEIGHTED CENTROID)
    8 = WPGMC (WEIGHTED CENTROID OR MEDIAN)
    9 = INCREMENTAL SUM OF SQUARES
   10 = HOMOGENEITY CLUSTERING (I,D:5)              ? : 5
    ORDER OF OUTPUT ASSOCIATION MATRIX (I,D:0=NONE) ? : 0
    USE ADJACENCY CONSTRAINT (Y/N D:N)              ? : N
    BETA (F,-1.0<= BETA <1.0,D:0.)                  ? :

.....FUSE: WORKING
```

The following table is the history of the fusions: which objects and groups fuse at what level of association. Rather than closely examining this table, a graphical representation of the process is created using DEND -

```
 08/07/89  14:53:36.17 FUSE  RANDOM DATA SET FROM RAND OPTION

 FLEXIBLE UPGMA OR GROUP AVERAGE FUSION WITH BETA = -0.10

 GROUPS          FUSION GROUPS          NEW GROUP   LEVEL  INCREMENT  STRESS
 ------          -------------          ---------   -----  ---------  ------

     9  ROW    3(    3)+ROW    5(    5)=GP(    3)- 0.241     0.000     0.00
     8  ROW    4(    4)+ROW    9(    9)=GP(    4)- 0.270     0.289E-01 0.00
     7  ROW    3(    3)+ROW   10(   10)=GP(    3)- 0.280     0.999E-02 0.00
     6  ROW    3(    3)+ROW    8(    8)=GP(    3)- 0.299     0.192E-01 0.00
     5  ROW    4(    4)+ROW    7(    7)=GP(    4)- 0.333     0.344E-01 0.00
     4  ROW    2(    2)+ROW    4(    4)=GP(    2)- 0.369     0.355E-01 0.00
     3  ROW    2(    2)+ROW    3(    3)=GP(    2)- 0.397     0.286E-01 0.00
     2  ROW    2(    2)+ROW    6(    6)=GP(    2)- 0.405     0.737E-02 0.00
     1  ROW    1(    1)+ROW    2(    2)=GP(    1)- 0.498     0.936E-01 0.00

 STRESS THRESHOLD= 0.917E-40   AVERAGE INCREMENT & STRESS :  0.286E-01 0.00
```

The Dendrogram

DEND reads the fusion table and displays a graph called a dendrogram. This provides a diagram that gives the *history* of clustering. The dendrogram shows that object 3 fused first with object 5 at the association value of 0.2409. The next fusion was between objects 4 and 9 at the value 0.270. This diagram displays the relationship between all pairs of objects and groups of objects. It is a fundamental tool for interpolation of data *structure*.



DOCUMENTATION

STRUCTURE

The documentation for **PATN** comprises two hard-copy manuals and the on-line help. The manuals include:

1. The Users Guide (this document) that is designed to provide an overview of the operation of the package.

2. A Technical Reference containing details concerning each **PATN** command with headings -

. outline
. references
. options.

The on-line documentation is activated by entering a question mark in response to any prompt from **PATN** in any module.

REFERENCES

Overall, the documentation is pitched at the user who has a little knowledge about computers, a good understanding of their data and a basic comprehension of Pattern Analysis. It is recommended that one or two of the following texts should be scanned before any detailed analysis using **PATN** is performed.

Anderberg M R (1973). Cluster Analysis for Applications. (Academic Press: New York) 359p.

Everitt B (1980). Cluster Analysis. (Heinemann Educational) 136p.

Clifford H T and Stephenson W (1975). An Introduction to Numerical Classification. (Academic Press: New York).

Romesburg, H. (1984): Cluster analysis for researchers. Lifetime Learning publications, Belmont, California, 334p.

Sneath P H A and Sokal R R (1973). Numerical Taxonomy. (Freeman: San Francisco) 573p.

Journal of Classification. Springer International. Published by the Classification Society of North America. 1984+

The documentation is designed to convey basically how all the algorithms and options operate. To some extent, much of the theory of Pattern Analysis is therefore contained in the Technical Reference. What is much more difficult, is to convey HOW the package and it's many algorithms and options should be used. Manuals are not the best means of communicating this. An expert or knowledge based system would be an alternative (if I had time).

Using **PATN** as a 'black-box' is not recommended. There is no substitute for at least scanning a number of the texts listed above and the references listed in the Technical Reference.

On-line documentation

PATN's on-line documentation is obtained by entering a question mark (?) at any prompt. The nature of the response depends on where you are in **PATN** and what system you are running it on. On UNIX and VMS systems, help for say HIST is evoked by

```
HIST?

HIST provides UNIVARIATE statistics and histograms of any or all rows or columns of
your data matrix. The histograms for each selected row or column can be printed with
any number of bars. Univariate information listed in addition to the histogram
includes:

       1.  Number of Values,
       2.  Minimum
       3.  First Quartile,
       4.  Median
       5.  Mean
       6.  Third Quartile,
       7.  Maximum,
       8.  Inter-Quartile Range
       9.  Average Deviation,
       10. Standard Deviation
       11. Range
       12. Sum
       13. Number > 0
       14. Skewness
       15. Kurtosis
```

At the menu level, on DOS systems, an outline of the alternative modules is presented. For example, from the pre-processing menu:

```
The preparation or pre-processing section covers the type of activities that are
usually done prior to the real (pattern) analysis. This includes the specification
of data parameters, the input, display and transformation of data and simple
statistics.

For convenience, the preparation modules are themselves broken into three functional
groups; input & output, statistics and display and data manipulation. The
classification is not rigid but is designed to assist in the navigation of PATN.
Using an analysis module for pre-processing data in some circumstances, may be
acceptable and necessary.

Unless you are a gun programmer, you will have to use the input/output modules to
specify data parameters and get data into and out of PATN. Modules such as PRAM,
DATN and often LABN and ASON must be used for this. RAND is for lazy analysts or
those wishing to test various PATN algorithms.

The statistics and display section are designed to check data integrity. HIST and
SCAT are of more use for continuous attributes while SCAN is applicable for presence
/ absence data.

The data manipulation modules basically alter data in some way. They may recode it,
eliminate it, sample it or mask it.
```

To get information about a question or prompt, enter a '?' by itself. For example, typing a question mark to the first prompt in RAND produces-

```
    First ROW to be generated (I,D:1,0=EXIT)        ? :

If you consider the data being generated in a tabular form (similar to a
spreadsheet), this value is the TOP row in the segment of the data being
generated. Entering a '1' to place values into the first row should be
done in one of the cycles otherwise rows 1 to the FIRST row you nominated
will be filled with missing values.

Each CYCLE of RAND creates a table of data with the selected statistical
criteria. There can be as many CYCLES as required to create the desired
data matrix. The resulting data matrix will range from the top-left
row-column (1,1) to the largest row and column numbers selected (n,m).

Four values determine the size and location of the block, the top-left row
and column and the bottom-right row and column. The three other parameters
are requested in sequence.

.....Please re-enter parameter to the last prompt...? :
```

FEEDBACK

Any comments on the documentation will be gratefully received. Address any correspondence to:

Lee Belbin
Blatant Fabrications
43 Harpers Road, Bonnet Hill,
Tasmania, Australia 7053

Phone: +61 3 6229 1910

GLOSSARY

This is an accumulation of many of the terms common in Pattern Analysis. If there are any additional terms that would be useful to add to this list, write me a short note and I will include it in the next release.

ADJACENCY. Objects, as areas, regions or polygons that are spatially next to one another, contiguous or share a boundary.

AGGLOMERATION. The process whereby individual objects are accumulated into a single group containing all objects.

ALGORITHM. The concise definition of a method for the solution of a specific problem that facilitates translation into a computer program.

ALPHANUMERIC. Characters that can be either alphabetical or numeric. In most contexts, all printing characters on a standard QUERTY keyboard are alphanumeric.

ATTRIBUTE. The variables used to describe the set of objects in the dataset. These usually form the columns of the data matrix, but may form the rows if an analysis of attributes is required.

ASSOCIATION. The general term in this document used to cover all the measures or coefficients of similarity, dissimilarity, difference, distance, proximity or affinity. The default type for **PATN** is a dissimilarity measure where the value zero (0) implies absolute equality and the value one (1) implies maximum dissimilarity.

ASYMMETRIC. Usually in relation to a matrices of association values, where the values of the lower left triangle of the matrix are not a mirror image of the upper right triangle. See SYMMETRIC.

BATCH. In computing, a mode of running a job where it is self contained and independent of a terminal.

BINARY. A term used as a synonym for presence/absence data. The term 'presence/absence' should be used in preference to binary.

CLUSTER. A natural or artificial grouping of objects with some implied or assumed affinity.

COPHENETIC CORRELATION. Pearson's Product Moment correlation coefficient between the original association values and those associations as derived usually from a hierarchical clustering of objects.

DEFAULT. A value or string that will be used if no data are entered in response to a prompt for **PATN**.

DELIMITER. A computing term used to denote the characters that are used as separators between values in an input or output record. For example, if commas (,) are used to delimit values, then the values are said to be comma delimited, meaning the separate values are separated by a comma.

DENDROGRAM. A diagram representing the history of the successive binary fusions (two objects or groups forming a single group) or dichotomizations (one group split into two components). A tree like structure with a single root representing the complete set of objects with branches representing objects or a group of objects.

DICHOTOMIZATION. The splitting of one group into two groups.

DIMENSION. A reference line in space initially corresponding to each of the attributes in a dataset, but applying equally to a set of axes as derived from ordination methods.

DIVISIVE. The process of dividing one group into successive sub-groups. Opposite to agglomerative.

EXPLORATORY DATA ANALYSIS. The technique of exploring data, looking for structure or displaying data in a form where its 'features' are more readily discernible. This is the purpose of **PATN**.

EXTRINSIC. An attribute that was not used in the analysis. Opposite to intrinsic.

FLOATING POINT. A storage-type encoding used by FORTRAN to store values having a real or implied decimal point.

FORMAT. A template or a set of rules for the arrangement of information.

FUSION. The joining or amalgamation of two objects or groups of objects.

HEURISTIC. A rule of thumb that is often used repetitively to progress from some starting configuration to a goal.

HIERARCHY. A structure showing nested grouping; where a group at any intermediate level of the structure is both a part (daughter) of a larger group at a higher level and author (parent) of other groups at lower level. For example, an organisation chart with a single chairman at the top and many workers at the base.

INTEGER. A storage-type used by FORTRAN for storing whole numbers (meristic values).

INTERACTIVE. A mode of computing where a program interacts with the user at a terminal.

INTRINSIC. An attribute that contributed to an analysis. Opposite to extrinsic.

INTERVAL. The third of the four scale types used to describe the coding of attributes where the interval between value on the scale are significant. Interval scale attributes also imply that there is nothing special about a value of zero. Interval scales imply that the difference between 100 and 200 degrees Fahrenheit is the same as the difference between 500 and 600 degrees Fahrenheit or

600-500 = 200-100

ITERATION. A repeating logical sequence of operations, each complete sequence of which converges to a specified goal.

LOGGING. The process of recording the options selected and the parameters entered by a user in a file called a log file.

MATRIX. A logical and consistent arrangement of data values where the position of values implies additional information.

MERISTIC. Whole or integer values such as counts. Meristic values can take the values 0, 1, 2, 3, 4 ....infinity.

METRIC. A class of association measures that conform to the following rules:

1. The distance between an object and itself is always zero.

2. The difference between two objects is the same, regardless of viewpoint.

3. Given three points forming a triangle of distances, the length of any side is less than the sum of the remaining two.

MINIMUM SET. This is a term that is used to define a reserve selection algorithm developed by Margules, Nicholls and Pressey (see Technical Reference). The algorithm that is implemented in PATN attempts to determine the minimum number of objects that are needed to sample each attribute (species) a given number of times. There are a number of options available.

MINIMUM-SPANNING-TREE. A network algorithm that is specified by forming a complete linkage (joining all objects) where the total length of the connections is minimal and where no loops or circuits occur.

MONOTHETIC. The contribution of a single attribute when used to agglomerate or split a group. Opposite to polythetic.

MONOTONIC. A series of values that show a consistent increase or decrease. Tied values are usually permitted. For example the values -

1 3 4 5 5 6 8 10 12 16 21 99 200

show a monotone increase, whereas the values -

1 3 4 5 4 3 1 5 6 77 1 8

do not.

MONTE-CARLO. A form of statistical test where the significance of an observed test statistic is assessed by comparing it with a sample of test statistics obtained by generating random samples using some assumed model. If the model assumes that all orderings of the data are equally likely, this implies a randomization test with random sampling of the randomization distribution. PATN contains such tests for attributes in an ordination (MCAO), ordination dimensionality (MCSSH) and the significance of a set of groups of objects (ASIM).

MULTIVARIATE. Using more than a single attribute (variable).

NETWORK. A set of connections between objects.

NOMINAL. The lowest (in terms of quality) of the four scales used to code attributes and where values are limited to embody the concept of 'difference' and 'identity'. For example, colours such as red, blue and green, while being coded as the values 1, 2 and 3, have no suggestion of red > green > blue (3 > 2 > 1). The only thing that can be determined is that red, blue and green are different. **PATN** cannot generally accept this scale as is. It must be recoded into a number of RATIO scale attributes. Using the above example, the THREE new attributes would be RED, GREEN and BLUE and the possible values on each would be not red (0), red (1), not green (0), green (1), not blue (0) or blue (1).

NUMERICAL TAXONOMY. Taxonomy is the process classification, the term usually applied in a biological context.

OBJECT. The basic unit to be analysed by **PATN**. Objects usually form the rows of the data matrix while the attributes form the columns.

OPERATING SYSTEM. The master program running on all computers that forms an efficient interface between the hardware (the physical aspects of the computer) and the user.

ORDINAL. The second in order on the scale of attribute coding where different values on the scale can be considered either 'greater than' or 'less than'. For example 'big', coded as 3 is GREATER THAN 'medium', coded as 2.

ORDINATION. The general term covering all techniques that attempt to condense information associated with the set of attributes to a limited number of new attributes.

PARAMETER. A value, character or character string that is used to modify an action. In the case of **PATN**, to modify the action of a command.

PARSE. A computing term meaning to scan a string of characters in search of a particular sub-string. For example, some **PATN** COMMAND PARAMETERS are PARSED in search of the characters 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9.

PATTERN ANALYSIS. The term I generally use to cover all techniques that search for patterns in data. Other terms covering this area include exploratory data analysis, numerical taxonomy and cluster analysis

POLARITY. A term introduced by me to suggest that data values show differential weighting depending on where they are in the scale. For example, differences between values low on the scale (2-1) are often assumed to be less important than the same differences high in the scale (100-99). This is suggesting the attribute is RATIO. Association measures themselves may be considered ORDINAL or RATIO in response to data.

POLYTHETIC. The process where many attributes contribute simultaneously to the splitting or merging of groups.

PROFILE. An attribute type where a single value is replaced by a set of values having some order dependency. For example, temperature could be either a single attribute as a set of environmental variables or expanded to a profile if, for example, monthly temperatures were available.

RATIO. The highest scale of attribute coding where the ratio of the difference between values is significant. For example the value 3 is 3/2 times larger than the value 2. This scale implies a meaningful zero value.

RECORD. A single line of information as seen by the user.

REVERSAL. The situation in hierarchical clustering where monotonicity of the successive levels of association fails. This situation occurs with the agglomerative strategies of centroid and median. What this implies, in this case is that, due to fusion, a new group is now closer to some other group than either of the two sub-groups that formed the new group.

SAHN. Sneath and Sokal Acronym used to describe 'Sequential, Agglomerative, Hierarchical, Non-overlapping' methods. These form one class of cluster analysis.

SERIATION. An ordination technique operating in a single dimension. The result of seriation is some meaningful ordering of objects.

STANDARDIZATION. The procedure of re-scaling data values such that they all conform to a constant formula and where at least some other values in the matrix determine each new value.

STRESS. Traditionally referred to in an ordination context as the difference between the original (input) dissimilarities and the distances as measured in the ordination space. The concept can however be applied to classification, and to a lesser extent, network analysis.

STRUCTURE. In the context of Exploratory Data Analysis, the arrangement of 'information' within a set of data. From this point of view, data can be thought of as 'structure' and 'noise'.

SYMMETRIC. As applied to matrices, a situation where the lower-left triangle of values is a mirror-image of the upper right values. Opposite of ASYMMETRIC.

TRANSFORMATION. A mathematical manipulation of data where each value is altered according to an overall formula and independent of any other values in the matrix.

TRANSPOSITION. The process of exchanging rows for columns and vice-versa in a matrix.

UNIVARIATE. Using a single attribute (variable).

VECTOR. A row or column of a data matrix forms the co-ordinates of the end point of a vector in multi-dimensional space.

COMMANDS

STRUCTURE

Commands in **PATN** correspond to the module names. The modules are stand-alone programs that broadly correspond to a particular component of pattern analysis. How each of the modules are activated will depend on the version you are using. On UNIX and mainframe systems, commands are entered in response to the standard **PATN** prompt-

PATN<

If you are using the menu system on DOS/WINDOWS systems, a single letter (that is usually a part of the module name) is pressed to activate the relevant module.

Commands for the Operating System

On UNIX and other mainframe versions of **PATN**, a special character is used to execute operating system commands. **PATN** detects a dollar character (**$**) in the *first position* of any command, and assumes that you want the command sent to the operating system for immediate execution.

PATN will not *parse* any character beyond the first so any errors in the string will be submitted to the operating system as is, with the same results as if you submitted it outside **PATN**. In addition, the commands have to be self contained, requiring no further input to the operating system. The intent of this feature was to allow file manipulation (copying, re-naming, deleting) and other operating system commands that would be useful in the context of **PATN** (show date and time...etc.). An example of such a command would be:

*__PATN__< $COPY FRED.DAT GEORGE.DAT

which would copy the first file to the second file.

Providing Comments with Commands (!)

PATN scans commands until sufficient information is obtained. In the case of all *commands* and *command parameters,* a maximum 20 characters is allowed. The one exception is the *alphanumeric*-style of input. In this case, file names are assumed to end with a blank character, FORTRAN formats with a right parenthesis and titles are unconstrained.

Where either a constraint or a maximum number of characters can be anticipated, comments may be used on the same line after either commands or command parameters. For purposes of consistency and clarity, an exclamation mark should be used as a delimiter and the comment should be limited such that the overall record length is 80 characters or less.

Here is an example of an input file to **PATN** that has been annotated according to the above scheme-

```
ASO           !  7-JUL-86 ! 12:34:12    ! RANDOM DATA
1                         ! ASSOCIATION MEASURE
1                         ! 0=ZIP_1=TERM_2=PRINT
                          ! CLEAR TERMINAL 2 CONTINUE
FUSE          !  7-JUL-86 ! 12:34:33    ! RANDOM SET
5                         ! FUSION STRATEGY
0                         ! ORDER OF ASSOC. MATRIX
N                         ! USE ADJACENCY Y/N
0                         ! BETA VALUE UPGMA
1                         ! 0=ZIP_1=TERM_2=PRINT
                          ! CLEAR TO CONTINUE
DEND          !  7-JUL-86 ! 12:34:59    ! RANDOM DATA
10                        ! NO OF GROUPS 2B PRINTED
                          ! CLEAR TERMINAL
```

COMMANDS ORDERED BY FUNCTION

Preparation

Input

PRAM - Specify DATA and environmental parameters
DATN - Data reformatting
LABN - Input/creation of data labels
ASON - Association reformatting
RAND - Data generation by random variates

Data display

HIST - Histograms and univariate statistics of data
SCAN - Features of presence/absence-type data files
SCAT - Scatter plots of data (x-y, x-y-z)
TWAY - Two way table of data by classifications

Data manipulation

MASK - Masking and/or re-ordering data
MERG - Right merge of various files to data
SAMP - Various row/ column sampling strategies
TRNA - Transformations /standardisation's of associations
TRND - Transformation or standardisation of data

Data Analysis

Generating association between objects

ASO - Association measures between ROWS
GASO - Permit attribute grouping in association
TRNA - Transformation/standardisation's of associations
ASON - Histogram of associations

Classification

ALOC - Allocate ROWS to pre-defined 'seeds'
ALOB - Large version of ALOC (no labels used)
FUSE - Hierarchical agglomeration (generalised)
MDIV - Monothetic division by attribute association
PDIV - Polythetic divisive equivalent to UPGMA
TWIN - Hill's TWINSPAN (two-way indicator species)

Ordination methods

SSH - 'Semi-strong hybrid' multidimensional scaling
PCA - PCA (Tri-D + QR algorithm)
PCR - Orthogonal rotation of ordination vectors
DCOR - Detrended Correspondence Analysis/RA (DECORANA)
SERE - Seriation (1d ordination) based on parsimony

Networks

NNB - Nearest neighbour lists
BOND - Bonding lists on 1st/2nd neighbours
MST - Minimum Spanning Tree

Post-Processing

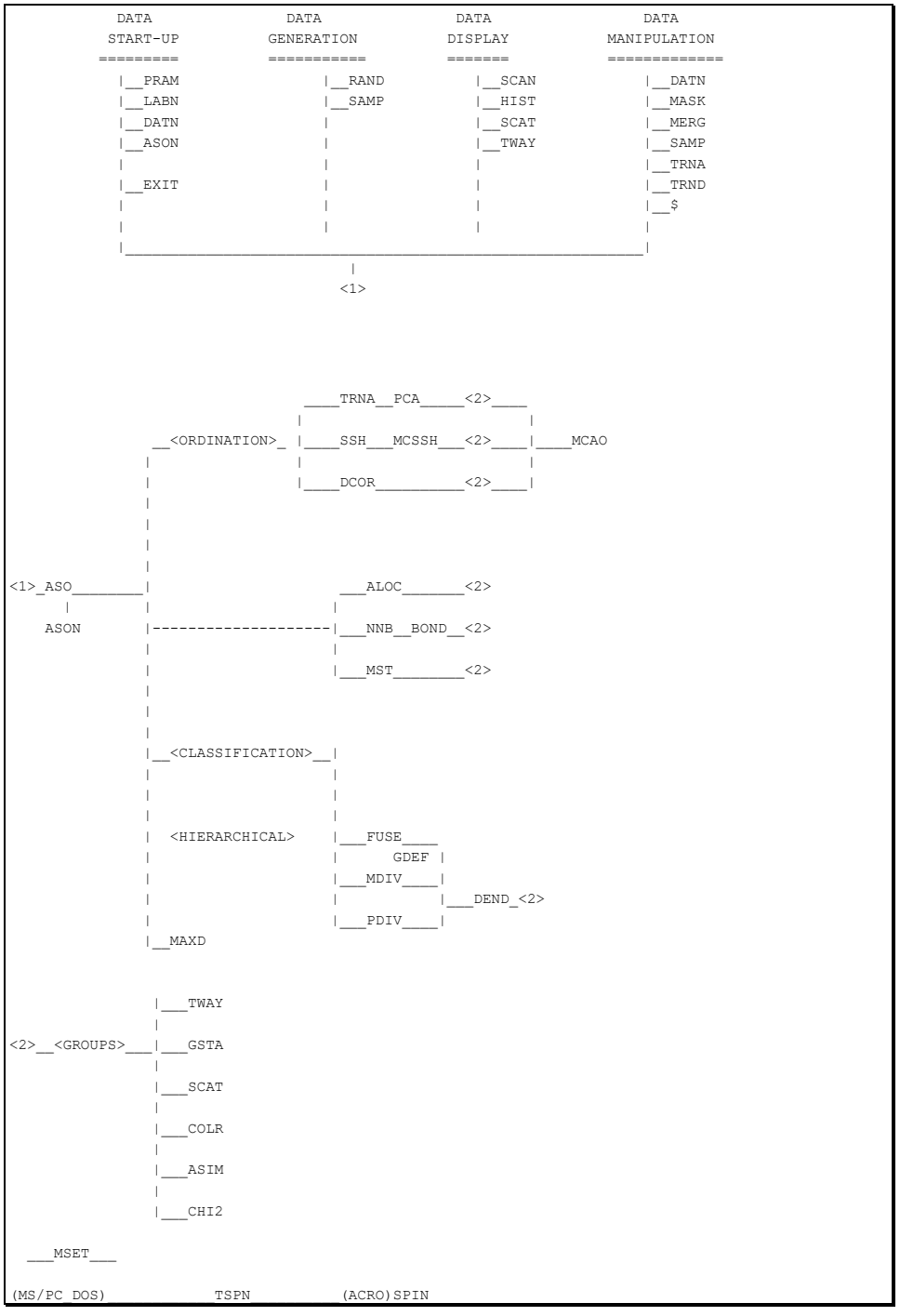
For classification

DEND - Dendrograms on line-printer
GDEF - Define groups / comparisons
GSTA - Statistics of groups or combinations
SCAT - Scatter plots of data (x-y, x-y-z)
TWAY - Tabulation of data by row/column groups
ASIM - ANOSIM (Monte-Carlo of groups)
RIND - Hubert/Arabie Rand statistic between two partitions
SENS - Sensitivity/redundancy analysis using Kruskal-Wallis
COLR - Simple mapping program (PC's) for groups
TSPN - Pre-processor for the Spin(tm) program
CHI2 - Simple Chi-square algorithm for attributes x groups

For ordination

PROC - Generalised Procrustean rotation
PCC - Regression of ordination vectors with attributes
SCAT - Scatter plots of data (x-y, x-y-z)
MCSS - Permutation tests to detect optimal dimensionality
MCAO - Permutation tests of attributes to ordination using PCC
TSPN - Pre-processor for the Spin(tm) program

COMMAND LINKAGES



COMMANDS ORDERED ALPHABETICALLY

PATN recognizes the following COMMANDS:

ALOC - Allocate ROWS to pre-defined 'seeds'
ASIM - Anosim randomization of -.aso across groups
ASO - Association measures between ROWS
ASON - Association matrix reformatting
BOND - Bonding lists on 1st/2nd neighbours
CHI2 - Chi-square of attributes to groups
COLR - Plot map of groups (spatial base required)
DATN - Data Input/Output
DCOR - Detrended Correspondence Analysis/RA
DEND - Dendrograms on line-printer
FUSE - Hierarchical agglomeration (generalized)
GDEF - Define groups / comparisons
GSTA - Statistics of groups or combinations
HIST - Histograms and univariate statistics of data
LABN - Input/creation of data labels
MASK - Masking and/or re-ordering data
MAXD - Maximally different sub-set of objects
MCAO - Randomisation tests of attributes using PCC
MCSS - Randomisation tests for ordination dimensionality
MDIV - Monothetic division by attribute association
MERG - Right merge of various files to data
MSET - Minimal sub-set given k replicates of attributes
MST - Minimum Spanning Tree
NNB - Nearest neighbour lists
PCA - PCA (Tri-D + QR algorithm)
PCC - Correlation of ordinations with attributes
PCR - Orthogonal rotation of ordination vectors
PDIV - Polythetic divisive equivalent to UPGMA
PROC - Generalized Procrustean rotation
RAND - Data generation by random variates
SAMP - Sampling an existing data file
SCAN - Features of 0/1 data files
SCAT - Scatter plots of data (x-y, x-y-z)
SERE - One dimensional ordination by parsimony
TABALO - Stand alone processing of TWAY results
TRNA - Transformation/standard. of associations
TRND - Transformation/standardization: data matrices
TSPN - (MS DOS) pre-processor for ACROSPIN(tm)
TWAY - Two way table of data by classifications
TWIN - Pre-processor for TWINSPAN

STOPPING

PATN may be stopped differently depending on the operating system in use. On PC's a "<Control-Z>" sequence is used (<Control-D> for UNIX). This is done by holding down the key marked "CTRL" and then pressing the "Z" key. On most system "<Control-C>" will abort the current task. If an option (eg. MASK) is operating, CTRL-Z or D will stop it and return to the **PATN** supervisor. If the stop sequence is used at supervisor level, **PATN** will be stopped. The tidy way to stop the supervisor is with the command "EXIT". If there is no prompt for input PATN may be aborted by CTRL-C.

PARAMETERS

Parameters can be either of two types:

1. *Environmental*. These are stored in the file **PATN**.PRM and define the current data and operating environment.

2. *Command*. These modify the action of **PATN** commands.

ENVIRONMENTAL PARAMETERS

Environmental parameters tell **PATN** the name and the nature of the file being analysed, and act as a logging switch that modifies the amount of tracking **PATN** is currently performing.

These parameters permit a number of different data files in various stages of analysis in a single directory. Any of these files can be activated by restoring saved parameters to the **PATN** *parameter* file *PATN.PRM*.

Most commands require information concerning the name, format and size of the file to be analysed. The first five parameters in *PATN.PRM* should normally be initialised by the user. Once initialised, **PATN** will update and generally maintain parameters to reflect changes made on the data file or optionally, the environmental values. The terms used below and the corresponding *parameter commands* will be used throughout the documentation when referring to these parameters.

The names in brackets in the following sections are the standard abbreviations that are used in various places in the manuals.

Title (TITLE)

A title of up to 80 characters of your choosing. It is important that the title is descriptive because it is used both to document your activities in the logging file *PATN.LOG* and to annotate output files. A sub-string may be inserted anywhere in the title by prefixing the title with a # immediately followed by a one or two digit number that refers to the column number where the following text string is to be inserted. The point of insertion is facilitated in this instance by the use of a 'ruler-line' in PRAM. This approach is useful when most of the title is retained and only a part is changed to reflect various stages of the analysis.

For example, if the current is:

KAKADU STRUCTURAL DATA 23-SEP-85

and is desired to append:

: CLASSIFICATION OF RATIO DATA

then the command in PRAM maybe:

#33: CLASSIFICATION OF RATIO DATA

making the completed and full title:

KAKADU STRUCTURAL DATA 23-SEP-85:
CLASSIFICATION OF RATIO DATA.

Data file name (ROOT)

DOS supports file names of up to 13 characters, other operating systems permit file names with considerably higher limits. **PATN** has been designed to accommodate file names of up to 43 characters. File names in PATN may include directories. The file name may contain a directory string as for example in DOS-

d:\fred\problem\mydat.dat

or UNIX:

usr/fred/problem/mydat.dat.

It is not generally recommended that paths or directories are included in the file name. It is more efficient to be working from a home directory that corresponds to the location of your data. The extension on the file (the letters after '.'), if omitted, is assumed to be 'dat'. It is therefore important that any ASCII data files are not named as '.dat'. This name will be used extensively by **PATN** as a base (termed 'root' in **PATN)** for all files produced. The number of files that **PATN** can produce in a comprehensive analysis may be 50 or more (most are small).

Number of rows in the data matrix (N)

This refers to the number of rows in your data matrix. This normally corresponds to the number of *objects* in the data file nominated above. A number of **PATN** modules assume that the rows of the matrix correspond to the objects; they are of primary importance. In some circumstances, the matrix may be transposed so that the number of rows in your datafile may be referring to the number of attributes.

Number of columns in the data matrix (M)

This refers to the number of attributes in your datafile. If the matrix is transposed, the is value may however correspond to the number of objects.

Number of row groups defined (NRG)

This is the number of groups of rows currently defined. This value may either be the number of groups of objects or attributes; it depends on the orientation of the datafile. A number of commands will result in the automatic alteration of this parameter. You may however alter them manually to suit any requirements you may have. NRG will be initialised to zero.

Number of column groups defined (NCG)

This is the equivalent parameter for column groups as defined above for row groups. The only **PATN** command that will automatically alter this parameter is DATN option 10 (data transposition), however you may alter it to suite any requirements you may have.

Missing DATA

PATN will handle most missing data in a logical fashion. If missing data is found, **PATN** will generally skip it and accept what is left. For example, it will skip a missing value of an attribute when comparing objects in ASO when either data value being compared is missing.

The default missing value on **PATN** initialisation is -9999. If you want another value to be used, use PRAM to alter the default. Do not use '0' as a missing data value!

Logging

A logging facility is included in **PATN** to enable various levels of recording of activities to take place. Recording is always directed to the file **PATN**.LOG and in some case the terminal. There are three different levels (intensities) of logging:

0 = no logging
1 = moderate (module level)
2 = complete (keystroke level)

With *no logging* nothing is written to the file **PATN**.LOG. *Moderate* logging echoes all parameters to the terminal and all **PATN** COMMANDS to the log file.

Complete or detailed logging echoes all parameters to the terminal and all *commands* and *parameters* to the log file. It is the option to use if you desire to maintain detailed tracking of your activities in **PATN**. This is highly recommended if you are serious about maintaining the maximum information about data and analyses.

Complete logging is useful for generating *batch* input to **PATN** and for detailed tracking of previous **PATN** sessions. Take note that the log file is maintained across **PATN** sessions and should be purged at appropriate times. The utility *LOG2B* is designed to read *PATN.LOG* and create a batch procedure.

When you know **PATN** well, setting the logging parameter to zero will result in faster response. '1' should be used when you're not in a hurry and a basic record of activities is useful. Setting '2' is of most use for creating batch file entry to **PATN** or tracing problems.

While getting accustomed to **PATN**, set the logging parameter to the value 2. This will maintain a record of all key-strokes during multiple **PATN** sessions. If any errors occur, the log file can be used to help determine the nature and the cause of the problem. It would be useful to see a copy of the log file if you contact me with problems with **PATN**.

With detailed logging, *PATN.LOG* will not only contain all key-strokes, it will also append annotations to each *command* parameter. If you use a small dataset during an interactive session with the logging parameter set to 2, the resulting *PATN.LOG* file may be renamed and edited to form an annotated input stream for the analysis of some other dataset. This is a useful feature for inexperienced users.

Saving environmental parameters

This option will save the *parameter* file *PATN.PRM* to the file *-.prm* where '-' = your root name. This effectively saves the status of a given parameter set for easy resurrection. For example, it is useful to maintain a transposed version of your dataset with its separate labels and parameter file.

Restoring environmental parameters

This option restores the *parameter* file *PATN.PRM* from any other file. By default, the extension is assumed to be PRM. This is useful when switching analysis to another set of data.

Summary

Environmental parameters should be initialised by you using the PRAM command, or by using RAND or DATN. After that they may be *optionally* modified by **PATN** programs to reflect a new status of the data or may be altered by you if required.

The options associated with the command PRAM that are used to list and modify the *environmental parameters* of **PATN** are:

```
Title - Description of analysis status........RANDOM DATA
Data File Name (extension assumed .dat).......RANDOM.dat
Number of Rows (Objects) in data matrix.......     5
Number of Columns (Attributes) in data matrix.    10
Number of Row GROUPS..........................     1
Number of Column GROUPS.......................     0
Missing Value.............................  -9999.

Logging (0=OFF 1=LIMITED 2=FULL).............     2
```

COMMAND PARAMETERS

Command parameters are those values and options that are required to guide **PATN** when such information cannot be determined from the environmental parameters or the data itself. *Command parameters* are the values that determine the nature of the operation performed in **PATN** modules. In ASO for example, the measure of association (1-17) is the only parameter required. In MST, no parameters are required, while SSH has more than half a dozen. The parameters can be thought of as sub-commands because they qualify the action of commands. There are four types of parameters that **PATN** will accept-

INTEGER (I),
INTEGER LIST (L),
FLOATING POINT (F),
YES or NO (Y/N) or
ALPHANUMERIC (A).

Default Values

PATN will always prompt the user, showing the type of input it is expecting. In addition, wherever possible, it will supply a default value. This is a parameter that has been considered as most appropriate under most circumstances. The default values are determined in one of two ways. If it is possible to do so from information available, **PATN** will decide on a context dependent value.

Second, for major options within some **PATN** commands, there are preferred pathways. These are, as far as the user is concerned, fixed default values. Defaults are not supplied when no reasonable guess can be made.

Numeric command parameters are always range checked. This means that **PATN** has decided on legal lower and upper bounds for each numeric parameter. If you exceed these limits an error message (last section of this manual) will be forthcoming and you will be requested to re-enter a valid parameter.

Default values are supplied to save unnecessary typing, not to provide an avenue to use the package as a black box. All options should be understood in context. Listed below are each of the 4 different types of PROMPTS and associated PARAMETERS that **PATN** will expect.

Integers (I)

Integers are whole numbers that can, in theory range from minus to plus infinity. They do not need a decimal point. They may include a minus sign. The prompt for *integer*-type input is:

(I,D:x<y)

where x is the *default* value that will be supplied if you press the *return* key and y is the maximum value the parameter can assume. In many cases, the upper bound is not listed. **PATN** can accept up to 20 digits and the number may occur anywhere in the 20 character positions following the prompt. Parameter input does not require any FORTRAN-type justification. The cursor will always be positioned ready for input. An example of a **PATN** prompt and associated *integer* input is:

ENTER THE NUMBER OF AXES REQUIRED (I,D:2) ? : 3

In this case, the user entered the value '3', overriding the default value of '2'. An important thing to remember for discrimination of *integer* and *floating point* values is that FORTRAN stores them in two different modes. While **PATN** will generally convert between the two for most parameter entry, it is wise to be consistent in using decimal points only with floating point values for parameters and data.

Integer Lists (L)

This is a style of input designed to save the user time in entering values that are in sequence when more than just a few *integer* values may be required. Such lists are to be found for example in row and/or column selection modules such as MASK, TRND, SAMP, HIST ...

There are four options that are always provided with this style of data entry. The user may choose the most convenient one. For example, for a few values or long contiguous runs of integers, keyboard entry is sufficient. If however, a long list of values is required, it is probably better to store the list in a file and direct **PATN** to read from there.

An example of the prompt is:

```
-----OPTION FOR INPUT OF VALUES

1  = Enter values from the terminal
2  = Read values from a file
3  = Accept all the values 1 -10  (I,D:1) ? : 1
```

PATN's response in this instance may be something like this:

```
ENTER VALUES (L) e.g.: 2 -4 6 e = 2,3,4,6, e = END

 ? : 1 3 5 7 10 -20 e<CR>
```

What is required here is a list of integer values separated by blanks or commas and terminated by an ' e' and a Carriage Return "<CR>". Contiguous (adjacent) values can be coded using negative values; for example

1 3 -8 15 -20 e <CR>

implies that the values 1, 3, 4, 5, 6, 7, 8, 15, 16, 17, 18, 19 and 20 are to be used. There are no *defaults* possible with this form of data entry.

If '2' was entered in response to the first option, **PATN** would request a file name and then accept values from that file with the same formatting requirements as noted above. If '3' was entered as a response to the first prompt, it will automatically generate the range of sequential INTEGERS; in this case the numbers 1, 2, 3, 4, 5, 6, 7, 8, 9 and 10.

Floating Point Values (F)

Floating point values can range in theory from minus to plus infinity and have a decimal point, either *implied* or *actual*. In practice, 32 bit (4 byte) floating point values can range from approximately $+-10^{-32}$. **PATN** will accept digits without a decimal point as *implying* that a decimal point is to be placed after the last digit. For example '1234' will be interpreted as '1234.'. As noted above however, it is wise to be consistent; use no decimal points with integers and always use decimal points with floating point values. The prompt for *floating point* parameters is:

(F,D:x.x)

where x is the default value supplied if the user presses the RETURN key. An example of a PROMPT and associated FLOATING POINT parameter entry is:

Enter the value for the threshold (D,D:0.8) ? : .7542

In this case 0.7542 was entered; the default not being used.

Yes or No (Y/N)

PATN often requires a simple *yes*, or *no* response. **PATN** will only respond favourably to either 'Y' meaning *yes* or 'N' meaning *no*. The **PATN** prompt for this style of input is:

(Y/N,D:x)

where the default is x: either 'Y' or 'N'. As an example of a **PATN** prompt and associated input:

Do you want to update the parameters (Y/N,D:Y) ? : Y

In this case the user entered the DEFAULT value ('Y'). Pressing the RETURN key would have had the same effect.

Alphanumeric strings (A)

Examples of the need for *alphanumeric strings* are titles, names of files, FORTRAN format statements and symbols for tables. Such input can be made up of one or more *printable* characters entered at the keyboard. *alphanumeric* literally means either *alphabetic* or *numeric*, but I use the term to also include characters such as '"!@#$%&()+<>?,./`~...'. These characters can be found on most keyboards.

The nature of what characters you use will depend on the context. Any printable character string may be used for *titles*. With regard to *file names* or FORTRAN *formats*, they must conform to the relevant rules defined by the operating system (DOS, OS/2, UNIX, VMS...). This manual contains a separate section on file naming conventions and FORTRAN formatting.

The **PATN** prompt for *alphanumeric* input is:

(Ax,D:y)

where x is the *maximum* number of characters permitted and y is the *default* string that will be used if you press the <*return*> key. As an example of this style of *prompt* and input:

Enter new data file name (A43,D:MAWSON.DAT)? : NAME.DAT

where a new file name, that had to be less than 44 characters long, has been entered by the user.

FILES

FILE NAMES

The number of characters permitted by **PATN** for file names is 43! The operating system may however limit this to a smaller number. For example, MS-DOS file names have a maximum of 13 characters. File names, given the length restrictions of the operating system, may also include devices, directories or paths. It is strongly recommended however, that you operate in the directory where your data is, not where **PATN** is! Paths or directory prefixes should not therefore be necessary, except to the \PATN directory where **PATN** code is stored (see installation notes).

Your data must be read into **PATN** before any data manipulation or analysis can be undertaken. The input and output modules and their function are-

PRAM: create the parameters describing your data
DATN: read (and write) data
LABN: read (and write) labels
ASON: Read (and write) association measures

The parameter file (*PATN.PRM*, *-.prm*), data file (*-.dat*), label files (*-.rlb*, *-.clb*) and association file (*-.aso*) are stored in *unformatted* form (binary or non ASCII). This implies that they cannot be typed, printed or edited by you. The modules above can translate the unformatted files to ASCII as well as from ASCII to unformatted.

The ROOT of the file name includes all characters up to the period (.) and should be *mnemonically* descriptive of file contents. The file *extensions* are the letters following the period (.). On DOS the extension can be up the three characters. On other operating systems, the limit is far greater. The extension is used to detail the style or type of file. **PATN** will use the *root* of the name used in the *environmental parameters* and append different extensions to this root to create new file names for storing the results from the execution of **PATN** *commands*.

For example the file name:

mawson.dat

identifies the file by its ROOT and its contents (DATA) by its extension (.dat). This style of file naming is common, with minor modifications across a number of different operating systems.

PATN will accept data from any legal file name. The root will not be altered at any stage by **PATN** but may be altered by you either to bring a new file to **PATN** or by renaming existing files. **PATN** will however create files with suitable extensions for most commands. A list of these appears later in this section. Take an example-

mawson

is the root, **PATN** for example adds:

.aso

creating a file named:

mawson.aso

which would contain a matrix of association values as created by the command ASO. The section entitled **FILE EXTENSIONS** details extensions used by **PATN**. An adjunct to this is that **PATN** will often assume that files with certain extensions exist for use as input. For example, the command DEND will, given the above example, assume the file:

mawson.fus

containing a fusion table existed. This is the file that results from hierarchical cluster analysis. Assumptions concerning input file extensions are meant to save unnecessary typing. In some instances, **PATN** will not be able to guess the appropriate name for an input or output file name, so it will request the information. In this situation, while a default file name may be supplied, any name can be used.

PATN will complain if required or nominated files do not exist. The only files **PATN** will *delete* are those used for *scratch* purposes or those you expressly permit to be over-written. Operating systems usually only allow a certain number of cycles or versions of the same file name. Some forethought is required if accidental overwriting is to be averted.

A special note in relation to data files is necessary. If data is modified, use a different name that reflects the changes for the output file. In this way, back-tracking, if necessary, is possible.

Automatic ROOT Additions

There are a number of situations in **PATN** where a new and complete dataset is created. For example, using TRND will usually result in some alteration to the default dataset (that pointed to by the parameters in *PATN.PRM)*. What should the new dataset (parameters, data and labels) be called? If it was given the same name as the original (input) dataset, this implies that the original will be overwritten. Not a wise move. **PATN** tries to make life easier by appending a single character to the ROOT of the input file name. In the case of TRND, the character used is 's' (implying standardisation; one of the operations of TRND). For example, if the input to TRND was 'FRED', TRND will request the name of the new (binary) output file as 'FREDS.DAT'. If you accept this default, parameters and labels as well as the datafile will be give the root 'FREDS'.

Other examples can be seen in ALOC where the inter-group associations and centroids are stored in a file with 'g' appended to the root. Similarly, DATN (option 8) will append a 't'. The number of other examples are growing as I get time. As you may have gathered, I believe that this type of operation is efficient and minimises mistakes.

FILE STRUCTURE

ASCII files

Wherever possible, **PATN** annotates the first few records of ASCII files with:

1. the current *title* as in the *parameter file* (**PATN**.PRM)
2. the *date* and *time* of writing the file
3. a *heading* showing details of the file *contents*.

PATN will use *row* and *column* labels to annotate output whenever it can. In most circumstances, in addition to labels, the sequence numbers associated with the rows and columns will be used. The standard format for labels and corresponding sequence numbers is:

A-LABEL (12345)

Where some compression of output data is required for formatting, the sequence numbers are dropped.

Unformatted Files

In some cases, the files resulting from some operation will be in binary/unformatted format. This means that the contents of that file cannot be listed or printed. For example, **DATN** will create unformatted new data, parameter and label files. None of which are displayable. If these files were to be typed or printed, unpredictable things would happen because the screen or printer may interpret some of inevitable control sequences as display commands when they are not.

SPECIAL FILES

The parameter file PATN.PRM

The file named *PATN.PRM* will be read after entering most **PATN** commands. This file contains the **PATN** *environmental parameters* in unformatted form. It *must* be initialised by using PRAM. This will need to be done before most other **PATN** commands can be invoked. The exceptions to this are the commands for generating data. In this case, **PATN** will also generate the environmental parameters in the parameter file and the associated row and column label files.

The *environmental parameters* inform **PATN**, and you, of the current data file name, contents and status as well as what the current level of logging is. The *contents* and *format* of this file are as follows:

. A TITLE,
. the current DATA FILE NAME,
. the number of ROWS (OBJECTS) in the data,
. the number of COLUMNS (ATTRIBUTES) in the data,
. the number of ROW GROUPS currently defined,
. the number of COLUMN GROUPS currently defined,
. the value to be recognised as MISSING data and,
. the level of LOGGING currently active.

The LABEL files -.rlb and -.clb

PATN will use *row* and *column* (object and attribute) labels wherever possible. These labels are stored in unformatted form in two separate files. ROW labels use the extension '.rlb' to your data file name and COLUMN labels use the '.clb' extension. For example, if the current data file, as nominated by the PARAMETER file **PATN**.PRM contained:

fred.dat

then the LABEL files would be:

fred.rlb for row labels and
fred.clb for column labels.

Labels may be created in a number of ways. A standard text editor can be used to create a set of row and column labels in a file prior to running **PATN**. This may be read by LABN. LABN can also be used to enter labels directly from the keyboard. It can even create a default set

ROW 1
ROW 2
ROW 3
.....ROW N

where N=number of objects, and

COL 1
COL 2
COL 3
.....COL M

where M=total number of attributes. The first three letters of the labels are user definable. As with the *parameter* file, the *label* files, once generated, are maintained and manipulated in accordance with **PATN** commands.

The logging file PATN.LOG

The file *PATN.LOG* can maintain an annotated list of all **PATN** *commands* and *command parameters*. This file is opened when a session is commenced. If the file doesn't exist, it is created. One record may be appended to the file for each command and command parameter entered to **PATN**, depending on the logging option set in PRAM.

With the logging parameter set to the value '2', *PATN.LOG* will contain three different types of information:

1. The date and time of starting **PATN**,
2. date and time of requesting each **PATN** *command* and
3. each of the user options with annotation by **PATN**.

This file can be used for two purposes:

1. To maintain a trace (log) of activities at various levels while in **PATN** and

2. To assist in the creation of a BATCH input for subsequent **PATN** runs.

With this information, errors can be traced and the style of analysis can be saved. Another feature is the ability to use the log file to record a macro. For example, RAND may be used to generate a dataset of the same size and nature as a real set. An analysis may then be run with all the steps recorded in the log file. This log file could then be edited and replayed with the one or more different datafiles. To achieve this, the logging file must be read by the stand-alone (not from menus) utility LOG2B. You should also copy the log file to another file for safekeeping, and delete the original.

An example of a log file *PATN.LOG* is shown below, showing an example of a simple analysis:

```
>PATN
     7-JUL-1986 12:34:06.67   ! ====NEW SESSION===
ASO               !  7-JUL-86 ! 12:34:12    ! RANDOM DATA
1                             ! ASSOCIATION MEASURE OPTION
1                             ! 0=ZIP_1=TYPE_2=PRINT
                              ! CLEAR TERMINAL TO CONTINUE
FUSE              !  7-JUL-86 ! 12:34:33    ! RANDOM DATA
5                             ! FUSION STRATEGY
0                             ! ORDER OF ASSOCIATION MATRIX
N                             ! USE ADJACENCY CONSTRAINT
  0.0000E+00                  ! BETA VALUE FOR UPGMA
1                             ! 0=ZIP_1=TYPE_2=PRINT
                              ! CLEAR TERMINAL TO CONTINUE
DEND              !  7-JUL-86 ! 12:34:59    ! RANDOM DATA
10                            ! NO OF GROUPS TO BE PRINTED
1                             ! 0=ZIP_1=TYPE_2=PRINT
                              ! CLEAR TERMINAL TO CONTINUE
```

FILE EXTENSIONS

PATN relies on the root or base of the file name as stored in *PATN.PRM*. A range of file extensions are added to this base. **PATN** will at times assume that files with certain extensions will exist. For example, FUSE assumes the presence of *-.aso*. If this file is not found in the default directory, FUSE will complain. If you do not generally override default file names, this type of error should be rare.

PATN appends standard extensions as defaults when creating all output files. If the module does not ask for an output file name, it implies that the file will have a standard name; one that you cannot/should not change! This file will normally be anticipated for subsequent input. For example, *-.aso* from ASO should not generally be renamed. There are some circumstances where it is OK. For example, if two association matrices are to be manipulated, you may opt to rename one to -.as1. The alternative is to change the root.

If the output file name is requested (with a default), any name with extension can be supplied, but some caution is required. For example, after transforming data with TRND, the output transformed data file is requested. You **may** supply *any* name/extension but if you do supply an extension, it should be *.dat*. If no '.' or extension is supplied, TRND for example, will add *.dat*. If you choose a name such as *trans.zzz*, subsequent operations on the new file may fail in circumstances where particular extensions are assumed. Standard extensions were designed for user efficiency. Some loss of flexibility is the price.

The essence is to stick with suggested filenames and extensions. If you want to save data in an unusual filename, fine, but be aware that it may eventually have to be copied or renamed to something more suitable at some later time.

A list of most of the standard extensions are listed below.

EXTENSION	MODULE(s)	IN/OUT	CONTENTS
.acd	TSPN	out	ACROSPIN(tm) input file
adj	ASON	in	adjacency coding
aso	ASO	both	association matrix
alo	ALOC	out	results
als	ALOC	out	seed seq. #'s
arc	DATN	both	Archive files
asc	ASON	both	ASCII associations
bon	BOND	out	results
cen	GSTA,ALOC	both	group centres
clb	LABN	both	column labels
dat	many	both	data file
dca	POST(DCOR)	out	DECORANA co-ords
den	DEND	out	dendrogram
dia	ASO,GOWC	out	diagonal file
fst	FUST	out	assoc. histogram
fus	FUSE	both	fusion table
gas	ALOC	in	metric-groupings
gcm	GDEF,GSTA	both	group comparisons
gdf	many	both	group definitions
gow	GOWC,PCA	both	Gower l-sym-matrix
gst	GSTA	out	group statistics
hst	HIST	out	histograms/stats
icl	LABN	in	stored row labels
icm	MASK	in	column mask
irm	MASK,TWAY	both	row mask
mca	MCAO	out	Monte-Carlo PCC vectors
min	MSET	out	minimal set result
max	MAXD	out	maximal different object subset
mst	MST	out	minimum span. tree
nnb	NNB,BOND	both	k-neighbour lists
pca	PCA,PCC,PCR	both	princ. components
pcc	PCC	out	PCA-att. correl.
pcr	PCR	out	rotated PCA
pdv	PDIV	out	results
prm	all	both	current parameters
pro	PROC	out	procrustes rotation
rin	RIND	both	cross-tab and rand index
rlb	most	both	row labels
scn	SCAN	out	p.a. data summaries
sct	SCAT	out	scatter plots
sed	ALOC	in	seed rows
ser	SERE	out	seriation results
smp	SAMP	out	duplicated rows
.ssh	SSH	both	ordination file
sym	SYMP	both	symmetric matrices
tar	PROC	in	target file
ult	ULTM	out	ultrametrics DEN, SSH
2wa	TWAY	out	two-way table

DATA

AN OVERVIEW

DATN can accept a variety of data formats-

. *ASCII*,
. *FREE* (space or comma delimited values)
. *COMPRESSED* (data with many zeros)
. RELATIONAL (indices of entries used) and
. *ARCHIVE* format (parameters, data & labels in one file).

Data can be thought of as forming a two dimensional *matrix* of values where the *rows* of the matrix are the objects to be analysed. **PATN** will analyse the objects in terms of the attributes, however, it is common practice to reverse the roles by transposing (see DATN) the data to obtain the inter-relationships between attributes. Because **PATN** currently generates association measures between rows of the data matrix, the analysis of attributes requires data transposition using DATN.

There is virtually no limit to the *nature* of objects and attributes that **PATN** can handle. Anything that can be described on the scales noted below can be meaningfully accommodated.

In addition to the values in the data matrix itself, **PATN** will assume a set of object and attribute labels. While a set of rows and column labels can be automatically generated by LABN, analysis of the results is simplified if a meaningful set of labels is entered.

ATTRIBUTE TYPES

Attribute is used in **PATN** to describe the suite of descriptive items that define or convey the qualities of the set of objects to be analysed. For example, if the objects are *cars*, a set of attributes may include colour, number of cylinders, horsepower, number of doors, cost, top speed, petrol consumption and so on.

The most useful method of understanding attribute quality and coding is presented in the following classification (see Anderberg):

1. Nominal
2. Ordinal
3. Interval
4. Ratio
5. Profiles

Nominal

The *nominal* scale, as its name implies, refers to a scale of measurement where the value assigned is consistent, albeit arbitrary. Brevity, combined with the fact that computers manipulate characters very poorly, promotes the use of numeric values where characteristics show NO superficial *order*. For example, colour may be coded as 'red', 'pink', 'green', but it is often simpler to use -

1 = red
2 = pink
3 = green .... and so on.

The important characteristic here is that 'green' is not greater (or less) than 'red'. Although 'green' may be coded as 3 and 'red' as 1, no ordering is implied. **PATN** will **not** accept *nominal* scale attributes as they stand, and unless recoded in DATN, will treat them as if they were *ratio* scale (see below).

Each different *code* for the *nominal* attribute must be recoded as a separate *ratio* variable. For example, the three colours noted above must be transformed into three new attributes called 'red', 'pink' and 'green' and replace the original attribute called 'colour'. An object may have either a '0' (zero) meaning 'no, I haven't got any of that colour' or a 1, meaning: 'yes, I have got that colour'. Note that each object, depending on its characteristics, may have either a single '1' where the new attributes are *mutually exclusive* or more than a single '1' where *mixtures* are permissible.

Note must be taken of the *number* of 'new' attributes generated from a single *nominal* attribute. The reason for this is that **PATN**, unless informed otherwise, will consider each attribute as having equal weight. If there were 20 original attributes and one of them was *nominal* in type and represented 10 different colours, the 10 new attributes representing the encoding of colour will be weighted the same as 10 original attributes and not 1. This may be undesirable.

Ordinal

The *ordinal* scale implies an *order* and nothing more. It implies that the coded value '3' is BIGGER than '1', but does *not* imply that it is three times larger than '1'. This type of coding usually occurs when representative values have been assigned to a set of class intervals. For example, measuring the exact height of a tree takes more time than saying 'it's bigger than 20 metres high'. The following coding is typical:

1 = below 1 metre high
2 = 1 to less than 5 metre high
3 = 5 to less than 20 metre high
4 = greater than 20 metre high

PATN will not know about your transformation table and will usually assume a *ratio* scale ('3' is three times greater than '1', and '4' is twice '2'). There are two things that can be done. Firstly, you may do nothing and be willing to live with the fact that some groups in a classification may contain mixtures of small and large trees (ie. they were coded closer than they should have been). Secondly, you may consider the distribution of your classes (see the command HIST) and recode (TRND) the values to give a better estimate of class differences. For example, the mid-point of the class interval, i.e.:

0.5 = below 1 metre high
2.0 = 1 to less than 5 metre high
12.0 = 5 to less than 20 metre high
30.0 = greater than 20 metre high

Interval

The *interval* scale goes one stage further than ordinal; it implies that '4' is '3' units bigger than '1' and '78' is also '3' units bigger than '75'. This *does* imply a *linear* scale but does not imply that the "0" value has any special significance, ie. it does *not* represent 'nothingness'.

An example of measurement on a *interval* scale is temperature in degrees Fahrenheit. There is nothing exceptional about 0 degrees Fahrenheit other than 'it's cold'. **PATN** does provide a number of *interval* association measures and some subsequent analysis methods (FUSE, SSH) provide options for interpreting association measures as on an *interval* scale.

For *interval* scale measurement, there is no difference between the comparison 1-2 and 101-102, both have a difference of 1 unit. Association measures such as the Gower metric (ASO) and the Minkowski series (includes Manhattan and Euclidean distance) can be said to be *interval*-type association measures since equal intervals will produce equal association values. Using the example above:

Gower Metric = (102-101)/Range = (2-1)/Range
Manhattan distance = 102-101 = 2-1

Use *interval* scale values with *interval* measures of association and analyse or transform the attributes using TRND to *ratio* scale.

Examples of interval attributes would include temperature, rainfall, slope and PPM nitrogen.

Ratio

The *ratio* scale adds the recognition of a true zero value to the *interval* scale. The *interval* scale implies that A is A-B times larger than B, while the *ratio* scale implies that A is A/B times greater than B. **IT IS THIS SCALE THAT PATN GENERALLY ANTICIPATES**. Most of the association measures that have been found to be 'superior' operate on data that is measured on this scale.

Using the previous example:

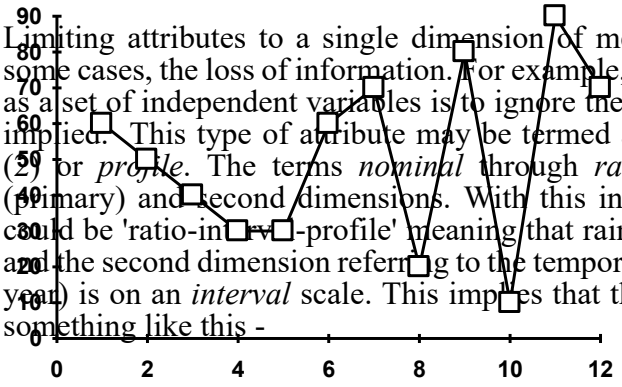
Canberra Metric = (102-101)/(101+102)
= .0097 is NOT= (2-1)/2+1 = .333

There are two special cases of the *ratio* scale, *presence*/*absence* attributes and *meristic* attributes. The former occurs when the coding is either present (1) or absent (0). The *meristic* scale includes all the positive integer values, that is: counts 1, 2, 3, 4, 5, 6 to infinity. These two special cases are best treated as *ratio* scale by *ratio*-type association measures and analysis techniques.

One way of understanding the significance of zero in the ratio scale is to consider presence/absence codes. While a '1' often means that a character or species is *present*, a zero may imply a number of things. For example, it may mean that the species or character was not seen, or that it was there and not recognised. In this case, the '1's are seen as *more reliable* than the '0's and are weighted accordingly.

Profiles

Limiting attributes to a single dimension of measurement scale invites, in some cases, the loss of information. For example, to consider monthly rainfall as a set of independent variables is to ignore the fact that a monthly *order* is implied. This type of attribute may be termed a 'two-dimensional variable' (2) or *profile*. The terms *nominal* through *ratio* still apply to both first (primary) and second dimensions. With this in mind, the rainfall example could be 'ratio-interval-profile' meaning that rainfall is a *ratio* scale variable and the second dimension referring to the temporal component (months of the year) is on an *interval* scale. This implies that the data should be viewed as something like this -



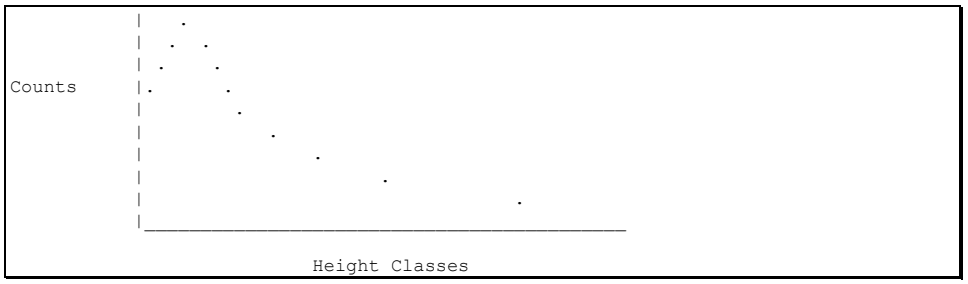
and not like this:

Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
60	50	40	30	30	60	70	20	80	10	90	70

It is implicit that each monthly rainfall is *not* a separate *ratio* variable but part of a *yearly profile*. To assume the former with this type of data may invite loss of information. For example, take one rainfall profile, and create another by shifting the first by a month or two. Summing the differences in monthly rainfall gives no clues to the fact that the profiles are identical except for a small translation.

Another example of a profile could be the number of trees and shrubs in various height classes. This would be referred to as a *meristic- ordinal-profile* in that the basic units (primary dimension) are integral counts while the height classes are probably not even.

For example:



Taken one stage further, we may equally wish to consider a three dimensional surface as an attribute type described by a set of 3 'lower-level' scale types. For example, a temperature surface over a geographic region may be described as an interval-interval-ratio surface. **PATN** is currently limited to 2d-profiles.

RE-CODING USING DATN

DATN provides an option for recoding of data from NOMINAL to RATIO scale. The module TRND has been designed for data transformations. Unlike, DATN which provides a specific transformation of NOMINAL scale attributes, TRND expects ORDINAL to RATIO scale attributes and provides for a wide range of transformations and standardisation's.

FORTRAN FORMATS

OUTLINE

The FORTRAN 77 language embodies coding to enable extensive control over the format of input or output. As **PATN** uses standard FORTRAN formatting conventions through DATN, ASON and LABN, some familiarity with these standards is required. Any mistakes in format specifications will usually lead to errors in the program or the data. In nasty cases, some files may require re-building. The worst scenario is where a mistake is made in an input format that is legal FORTRAN but results in data being incorrectly stored in **PATN**. Take heart, most internal operations in PATN use binary format so there is no chance for errors in translation. Just be careful with the number of digits and decimal places required when converting internal binary file to external ASCII or vice versa.

FORTRAN free format has values that are separated by blanks or commas. With this format, the probability of incorrect input is minimal. DATN for example will read free format data with a great degree of latitude. I recommend that free format be used for reading standard ASCII file into **PATN**. There is one drawback, don't leave any values out! With free format, once it gets out of sequence, values will be stored in incorrect locations in the file. If you have left one or more values out of the input ASCII datafile, DATN will run out of data (hit the end-of-file) and tell you so. It is often not easy to locate where the missing value is. DATN will not be able to help as data is streamed into arrays, one number at a time.

The trick with FORTRAN format is simplicity. FORTRAN uses formats in a consistent fashion and a little knowledge of its rules can save considerable time. For example, FORTRAN will re-invoke a format specification when there is more data to read or write and the end of the format is reached. In **PATN**, this means that if the data for each object is the same, a FORTRAN format for one OBJECT will suffice. For example:

```
(10F8.4)
```

means there are 10 numbers on each record, each taking up 8 columns with 4 decimal places assumed (the right-most 4 values) or whatever is after a decimal point. **NOTE**: a decimal point in the data takes precedence over the specified format. If the input data records are identical in format, the *record based* format will be sufficient. If not, then the format will have to be *object* based meaning that the complete format for the *first object* will have to be explicitly specified.

Neither style of format need take *explicit* account of the number of objects. The *record* based format need not even take account of the number of attributes *if*:

1. the same format is used on each record and
2. each new object starts a new record.

This is the easiest way to maintain data in ASCII format. It is achieved by finding the attribute that requires the greatest number of digits before and after the decimal place and using this for *all* attributes. Additional blank characters will result in wasted space in the file but the data will be simpler to input to most applications.

An *object* based format is required when the attributes of a single object require more that one record and the format is different on different records. An *object* based format may be required when the values of attributes would generate too much wasted space if a *record* based format was used. For example, the *input* format:

```
(60F1.0,/,4F10.6)
```

represents an *object* based format where each object has 64 attributes. It would be wasteful to put this data into '(8F10.6)' format.

JUSTIFICATION

Justification means to move values in a format field either to the right or left limits of that field. It is analogous to marbles being rolled up against the right or left end of a tube. FORTRAN will usually assume numeric values are *right justified*. If they are not, it will add the necessary trailing zeros so as to fill the field to the right. This has embarrassing implications. If you entered:

```
334
```

in the FIRST 3 columns of a 5 column field (format '(I5)'), FORTRAN will assume the value is 33,400 !

PATN stores most data in FORTRAN *floating point* (F-type) variables. If an *integer* type (I-type) format is supplied when an F-type is expected, an error will occur. This presents no problems because *integer* values can be read and stored correctly using F-type formats. **The opposite is not true**. For example, the integer value '3' will be read and correctly stored as '3.' using an F-type format while the floating- point value '3.1415926' would be read and stored as '3' using an integer (I) format. Hence the emphasis on F-type formats in **PATN**.

INTEGER VALUES (I)

These are *whole* numbers in the range minus infinity, through zero to plus infinity and have no decimal point. They are *right* justified within the field nominated. The FORTRAN format type is 'aIb' where 'b' refers to the number of digits or the width of the field in characters and 'a' refers to 'how many ?'. For example:

```
(5I4)
```

implies 5 fields of 4 columns (digits) making a record of 20 characters. An example of an input data record for the above format specification:

```
column number
12345678911234567892

   1  12 1231234 43
```

This would be read as the values:

```
1 12 123 1234 and 430
```

FLOATING POINT VALUES (F)

These are values with an implied or actual decimal point and range in theory from minus to plus infinity (but is actually limited in the range of values the computer storage can hold usually 32 bits these days). It is important to remember that *floating point* values are stored in a computer differently to *integer* values. The form of the F format is 'aFb.c' where 'b' represents the width of the field in characters, 'c' represents the number of implied or actual decimal places and 'a' represents the number of fields or values. An example of a *floating point Fortran format* would be:

```
(4F6.3)
```

This implies 4 values, each using a width of 6 positions or columns and having 3 decimal places implied. For INPUT, if there is no *actual* decimal point in the value, it is assumed that the value is right justified and the last 3 digits will represent numbers after the decimal point. If a decimal point exists however, it will *override* the format specification. As an example of the above format:

```
Column numbers
12345678911234567892123456789312345678 9

1.23     5.6      123456.45897
```

would be read as the values

```
1.23, 5.6, 0.0, 123.456, .45897
```

There is a quirk in the current definition of FORTRAN in relation to the output of data using F-type formats. Unlike input, when FORTRAN writes data using F-type format specifications, a decimal place *must* be written. This is sometimes a nuisance when trying to write (I1) style data (presence absence) but causes no other problems. See below for a method for circumventing this limitation.

ALPHANUMERIC VALUES (A)

As far as **PATN** is concerned, *alphanumeric* characters are all the single keys on the keyboard that result in *printable* characters. They consist of all the alphabetic, numeric and other characters that can be readily typed and recognised on a terminal. The form of the *alphanumeric format* is 'aAb' where 'b' is the number of characters and 'a' is the number of strings of length ‘b’. With FORTRAN 77, as used in **PATN**, 'b' is not required because it is determined by the actual declaration of the string within the **PATN** code.

An example of a *Fortran alphanumeric format* is:

```
(A20)
```

and the following string of 20 characters would be acceptable under this format.

```
 column number

12345678911234567892123456789312345678 94

This is a string of characters !+++-
```

ADDITIONAL OPTIONS

FORTRAN, in addition to the above formats, allows a variety of additional control to assist in formatting input and output. A general rule in relation to formats is to use the *simplest* format possible for both input and output. For example, instead of:

```
(F3.0,F4.2,F5.0,F2.0,F1.0,F5.1)
```

it would be more efficient in time to generalise the format to the lowest common denominator. In the example, the largest field size and number of decimal places. The above format would simplify to-

```
(6F5.2)
```

'Free'-format

FORTRAN provides for reading values that are delimited (separated) by either:

1. one or more spaces or
2. a comma.

PATN optionally reads most ASCII data in this form. In some cases, data preparation and entry lends itself to free formatting. For example, when entering data using a screen editor, attempting to position values in fixed columns is error prone. Using a comma or space as a delimiter is somewhat easier. An alternate source for this style of input is output of data from programs written using the computing language **BASIC**.

Spacing (X)

This is intended to read over character positions on input and to write blank spaces on output. The form of the spacing format is 'aX' where 'a' refers to the number of positions to skip. For example the format:

```
(5X,I4)
```

implies for input, 'skip 5 positions or columns and read a right justified *integer* value in a field of width 4 columns'. Using the above format, the following record is read as the value 789

```
 column number

12345678911234567892

     789
```

Repetition (n(....))

This is a useful feature for *repeating* a *grouping* of different formats. For example, the format:

```
(3(2F2.0,F6.2))
```

implies three lots of:

1. two floating point values (2 columns each) followed by
2. a 6 *column* floating point value with 2 decimal places.

For example, this would be useful to read the following data:

```
Column numbers

123456789112345678921234567893123456789 4

1.0.23.01 0.1.340.341.1.9870.1
```

which would be read as the values:

```
1, 0, 23.01, 0, 1, 340.34, 1, 1, 9870.1
```

Tabbing (TLn) or (TRn)

The tabbing feature of FORTRAN is useful for writing floating point values in what appears to be an integer form (getting around the problem of the forced decimal points on output). In this context, tabbing should only be used as an *output* form, not input. The form of the tabbing format is 'Tab' where 'a' can be either of the characters 'L' (left) or 'R' (right) and 'b' is the number of positions to move. Take the following example:

```
(F2.0,TL1)
```

As an output format this implies 'write the floating point value in two columns, the first being a single digit and the second being a decimal place, then tab back one position (over the decimal place). Output following this will then *overwrite* the decimal point achieving the same result as in using an I format.

As an example, the following 20 values (either zeros or ones were output using the format (20(F2.0,TL1))-

```
column number

12345678911234567891123456789312345678 94
10000100010000001011100001110000011000 00
```

Printer Control

FORTRAN output has a unique characteristic that may become a nuisance on some operating systems. The character in the *first* column of each line of a file is used to control the printer as follows -

```
' ' advances 1 line before printing the row (usual)
'0' advances the paper 2 lines before printing
'1' advances the paper to the next top of page
before printing
'+' do not advance one row before printing
```

These codes are a hangover from early IBM days of printer control. The consequence of 'carriage control' is that it is, for example, unwise to use:

```
(10F4.0)
```

when you know 3 figure values are possible (remember the decimal point takes one column), because the first digit on each output record will be used to control the printer. To overcome this problem, use something like:

```
(1X,10F4.0)
```

or simpler still:

```
(10F5.0)
```

HINTS

While data for **PATN** must conform to the prior specifications, *parameter* input to **PATN** options need not. The reason for this is that such parameter input is *parsed* (scanned) by **PATN** and the intention can usually be determined. It is wise however, when working with FORTRAN, to stick to the rules and use decimal points where indicated.

The data format assumed by **PATN** is largely *floating point* (F). If you have presence/absence data (1/0), there is a trick that will force FORTRAN *not* to write out a decimal point. This should only be used on *output*. To achieve this use:

```
(n(F2.0,TL1)).
```

signifies 'n' lots of 'TL1' meaning 'tab left 1 space', and thus each decimal point - except the last on the record - is overwritten. For example: format (80(F2.0,TL1)) writes the equivalent of (80I1) format. NOTE again, this should be used *only as an output Format*.

PROMPTS AND MESSAGES

PATN attempts to use a *standard format* for output, and anticipates a limited range of responses as input. The four forms of *parameter* input are covered in a separate chapter, while the *prompts* and *messages* used by **PATN** are covered below.

MASTER PROMPT

The nature of the prompting will depend on the operating system you are using. In DOS, a set of menus are used to help you navigate around **PATN**. If this becomes tiresome, as it probably will when you get to know the names of the various modules, you may simply run **PATN** from the DOS prompt. For example, typing

DATN

will run the data input and output module. Simple. If you are using UNIX or VMS, the names of the individual modules may be used as above, or the PATN front-end may be used by typing-

patn
PATN:<

In this situation, **PATN** is waiting for a *command*. The range of possible commands are listed in the chapter entitled **PATN COMMANDS** while the range of responses are covered below.

PATN commands

PATN is command driven in UNIX and VMS. Legal *commands* comprise a string of up to five characters used to identify a module. For example,

PATN:< RAND or
C:\PATNDAT> RAND

would initiate **PATN** to begin the procedure for generating data and associated environmental parameters based on statistical random deviates. Like most commands, RAND will request any necessary information that will be required to generate an output data, label and parameter set.

IMPORTANT: regardless of the operating system that you are using, if you want to run **PATN** in non-interactive or batch mode, then the command mode is used. For example, the DOS version is run in command mode via a batch file either created by you or by the use of the utility LOG2B on the log file *PATN.BAT*.

Commands for the operating system (UNIX)

Including a dollar sign in the first character position of a command, automatically flags the command for execution by the operating system. **PATN** is unable to check the legality of such commands; it leaves that up to the operating system. If it is illegal, the system is likely to let you know.

This style of command is limited to a single *record* (one line of characters), *not* multiple lines. In addition, the command itself cannot generate additional requests; it must be self-contained. As an example, under VMS, the following command copies a file *mawson.dat* to another file FRED.DAT:

PATN:< $COPY MAWSON.DAT FRED.DAT

Comments in Commands(!)

PATN ignores blanks and exclamation characters in the first character position of input to the master prompt in command mode. This allows comments to be embedded in command input to **PATN**. While this is not useful when using **PATN** in an *interactive* mode, it is a useful feature in *non-interactive* and *batch* mode, detailing what various commands and parameters were for. For example:

PATN:< ! I will re-do the analysis with the Kulczynski

PATN:< ! Coefficient and see what happens

PATN itself uses this feature when the *environmental parameter* governing logging is set to the intense level (2). When this is in effect, **PATN** will append comments detailing the nature of all *command parameters* to the end of the record containing the parameter itself. **PATN** uses an exclamation mark to announce 'what follows is a comment about the parameter to the left'. For example, if you entered the *integer* value '2' to a prompt, with the logging parameter set at 2, then a record such as:

2 ! This is the number of axes chosen

will be written to *PATN.LOG*.

LISTS OF OPTIONS

When **PATN** presents a list of 2 or more options it prompts at the terminal with the following format:

-----message:

where 'message' provides some notion of what the following list represents. With this style of prompt, the list will be *keyed* with a set of *integers* ranging usually from one to a maximum of twenty. The option is selected by entering the integer value (*command parameter*) corresponding to the desired option.

For example:

1 = Bray & Curtis measure
2 = Kulczynski measure
3 = Simple Matching Coefficient (I,D:2) ? : x

This list provides a choice of three of the association measures available under the ASO command. What is required in this case is the selection of the *integer* corresponding to the desired option (the default is two).

ADVICE

In this situation, **PATN** is not taking chances. The result of an action may not be obvious and **PATN** is advising accordingly. In addition, **PATN** will announce that it is working when nothing appears to be happening and **PATN** is reading, writing or calculating. Sometimes, the information that **PATN** supplies as advice will be required at some later step. The form of the prompt is:

.....message

WARNINGS

In a few places in **PATN**, the implications of certain actions may not be obvious to the novice. In this case, **PATN** uses the following form of prompt:

*****message

to alert you to a potential disaster. Take note! For example, DATN does not always produce labels when reading data into **PATN**. In a number of situations, **PATN** is incapable of figuring out all the possibilities and intents, so warns you to think about it yourself.

ERRORS

In this case, **PATN** has detected some type of error condition. Either **PATN** has got it wrong (hopefully rare) or you have. It is for example, a common mistake to have a mismatch between the *environmental parameters* and the data these parameters detail. In many cases, the error message only *indirectly* points to the cause.

The form of the prompt is:

>>>>>message

In some cases the error is fatal and the command will *abort* (*stop*). In this situation, the command is unable to be executed successfully. In other cases, **PATN** is may be able to carry on by requesting *correct* information. The errors associated with *files* are:

>>>>>END OF FILE IN FILE < >
>>>>>FILE NAME < > CANNOT BE FOUND
>>>>>ERROR IN READING FILE < >

where < > refers to the active data file name.

ANALYSIS GUIDELINES

While **PATN** contains a large number of analysis pathways, non-default options are used only rarely. This outline is provided as a basic exploration of data of a type that is not unusual in the sense of needing some recoding or transformations. A basic **PATN** analysis *should* consist of the following four segments:

DOCUMENTATION
PRE-PROCESSING
ANALYSIS
POST-PROCESSING

DOCUMENTATION

Once you have decided on using a particular **PATN** option, the associated *documentation* should be examined: either the Technical Reference or the on-line help. **PATN** prompts are generally somewhat brief. The documentation in the Technical Reference is provided as a more complete explanation of what each command parameter is requesting. At this time, the documentation is not suitable as a comprehensive treatise on the theory, rather it is a basic rationale to the algorithms and associated specifications required.

PRE-PROCESSING

Detailing Data

The first requirement is to nominate the **PATN** *environmental parameters* which detail the nature and amount of data is to be analysed. The name of the datafile as well as the number of rows and columns must be specified before **PATN** can be expected read your data. The only way to do this is to enter these parameters using the module PRAM. Select PRAM from the menu or typing it at the operating system prompt followed by pressing the <*return*> key will initiate the necessary few question to answer to accomplish this.

Reading Data

The module DATN **must** be used to get your data into **PATN**. The only alternatives are to use RAND to generate some or use ASON to read an association matrix that may have been calculated elsewhere.

Fiddling

The most common *pre-processing* option for data manipulation is masking (the module MASK). With presence/absence data, columns containing all *zeros* (no data) or a single 1 (not sufficient information for analysis) should be eliminated. If the dataset is large, a subset may be chosen either purposefully with MASK or by a variety of options using the command SAMP.

For non-presence/absence data, the module HIST may be useful to make sure that the data has been read into **PATN** correctly. It is possible to stipulate an input format that **PATN** will accept and use to read the data with, but incorrectly. Errors will not occur unless the format mismatches the data enough to produce a read error. HIST produces histograms and a variety of univariate statistics. Alternatively, SCAT can be used to produce bi-variate scatter plots and regressions based on pairs of attributes. For presence/absence data, SCAN forms a better alternative to HIST.

ANALYSIS

Association Scale

The heart of most Pattern Analysis methods is the estimation of association between pairs of objects. If association is poorly estimated, subsequent phases cannot always be expected to recover. A rule of thumb is to use the *Bray & Curtis* measure of ASO (option 1) when matches between higher values of attributes are *more significant* than matches between lower values on the same attributes. If the size of the value is unimportant and only the differences are of interest, use the *Gower* metric. To illustrate, consider the values on four objects and a single representative attribute:

	Attribute Value
Object 1	1
Object 2	2
Object 3	101
Object 4	102

If you consider the difference between Objects 3 and 4 to be less than the difference between objects 1 and 2, use *Bray & Curtis*, otherwise *Gower Metric*. If the *polarity* is reversed, that is objects 1-2 are deemed closer that objects 3-4, then the scale of this particular attribute may need re-coding. To do this use TRND option 11 (linear interpolation). Give the highest value to the current low value and vice-versa. The rationale is simple, attributes that have a distribution that is *skewed right* promote the weighting of higher values as being more significant because *matches* between high values are less likely. If the attribute is *skewed left*, reverse re-coding may be appropriate for a non-linear response of the association measure.

Another way of summarising this to use the attribute descriptors nominal, ordinal, interval and ratio. The Gower metric can be considered as an interval association measure because equal differences in the scale are treated equally. Measures such as the Bray and Curtis (Czekanowski) and Kulczynski could be termed 'ratio' because the 'distance' away from zero is now a significant factor in the generation of association.

Distance From Your Data

It is a good practice with set of data less than 1000 objects and attributes, to perform an analysis on both the *objects* and the *attributes*. Both classifications may be combined into a two way table (module TWAY) using transposition (module DATN) to exchange rows and columns of data (and labels). This imposes the results of computation back on the data where effects can be more readily evaluated. For the analysis of 'species-type' data, the association measure two-step (ASO option 6) is recommended. If the attributes are not akin to species counts or presence/absence, then use the same decision as above to apply either the Bray & Curtis measure or the Gower of ASO. If the attributes have mixed scales (see section on DATA), then some form of standardisation of the data by attribute will be required for the measure of association to produce meaningful results (use module TRND).

One Step?

Pattern Analysis should not be conceived as the application of a single technique such as UPGMA but in most cases, should consist of *one of each* of the categories:

Classification (FUSE or ALOC)
Ordination (SSH)
Networks (MST, NNB and BOND)

The different classes of algorithm are complimentary, showing different aspects of the data. They are not mutually exclusive. *Classification* techniques will, by definition impose grouping, whether it exists or not! It will also detect outliers which will adversely affect all ordination techniques. *Ordination* may detect natural clusters if they exist. Ordination also has then benefit of highlighting overall trends or gradients. Unlike the former techniques, *network* methods concentrate on local structure and therefore clarify relationships alluded to with classification and ordination.

A comprehensive Pattern Analysis should could combine all categories by overlaying for example, an MST with UPGMA groups on an ordination layout. The most comprehensive overlaying of results can be achieved with the PATN module TSPN ("to-spin"). This packages up a variety of **PATN** output (clusters, ordination, PCC and MST) and creates an input file to the ACRISPIN (tm) program. This program enables real-time rotations of three-dimensional structures defined by points and lines. Once you have seen ACROSPIN on data output from **PATN**, it will be hard to live without.

POST-PROCESSING

Why & Wherefore?

These options are designed to tell you *why* the analysis option provided the results as it did, as well as enhance the display of analysis methods. In some circumstances, it may be appropriate to ask **PATN** about a particular clustering or ordination that was generated externally. Such patterns may even have been generated subjectively.

Statistics & Plots

The two most common options are GSTA (group statistics) and SCAT. GSTA requires a set of pre-defined groups and provides a graphical discrimination between groups based on attributes. SCAT is used to effect with ordination results, plotting the spatial distribution, with and without attribute values as labels. COLR can be used to display groups in colour on a PC (not implemented in UNIX versions). COLR requires a set of x and y co-ordinates (longitude and latitude will do) and a set of pre-defined groups in either a *.gdf* or *.gav* format.

In some circumstances, you may need to test the validity of a clustering; ASIM does this. In addition, a pair of classifications may be compared using RIND. In this situation, you may have created the classifications using different methods, or using the same method on different attributes of the same objects. In the latter case, you could even subtract the two association matrices (module TRNA) and classify the resulting difference-association matrix.

Ordinations may be evaluated using PCC, MCAO and MCSSH. Similarly, two ordinations may be compared using PROC.

THE DETAILS

The following section lists the various *commands* with some of the important decisions that need to be made. The analysis suggested here is basic in the sense of not encompassing data with unusual characteristics. To gain a clearer understanding of the various commands used below, the relevant portions of the Technical Reference will need to be read. Default settings are used wherever possible.

Pre-Processing

Use PRAM to set-up all the parameters of the *data*. Take note of the default logging level and missing data value (do *not* use 0.0 for this value unless 0 really represents *missing* data: a very strange situation that is not recommended). If the data is some other form or requires re-formatting, see if the DATN options can be of use.

If some columns are all zero or a column contains a single '1' then MASK may be used (indirect masking) to eliminate rows or columns with sums or number of non-zero values less than a user-defined threshold. Possibly, you may like to view the data with HIST for ordinal-ratio data or SCAN for presence/absence data. Other possibilities include DATN for transposing or TRND for data transformation or standardisation’s.

Association

Generating an association measure between all pairs of objects is usually only viable and profitable when the total number of objects is less than about 1000. If the dataset is less than this, the standard *association-classification* steps outlined below are the best. For larger datasets, use non-hierarchical clustering as embodies in ALOC. For the *ordination* step use the inter-group association matrix to produce a display of the group centroids rather than each object in the group. Datasets above 500 or so objects *must* use group means rather than objects due to the inevitable time and memory requirements (unless you have a very fast UNIX system). Little is lost via this method because, with such as large number of objects, the ordination is usually cluttered anyway.

If all attributes are of equal weight and

The *higher* value attributes are significant (see ASO)

then use ASO (BRAY-CURTIS)

Otherwise

try ASO (GOWER)

Else if attributes are *mixed* in type or some re-evaluating of weighting is required then it is probably better to create separate data files using MASK. Each set contains the same set of objects, but with a consistent type of attribute. For example, one set may contain presence/ absence data where the *Czekanowski* option in ASO will be used. The other set may contain data more suitable for the *Gower metric* option of ASO.

Once ASO has been used on each set of data, TRNA should be used to range standardise and add the separate association matrices back together. Previous versions of **PATN** attempted to do this automatically but could not effectively handle the weighting of the variety of association measures. Well, PATN could handle a variety of problems but there were too many dangers for the novice.

TRNA (network option) may be used next to gain an upper limit for those values of association that have been under estimated (see Faith, Minchin and Belbin, 1987). Underestimation is considered to be operating with association values (Bray & Curtis, Gower, Kulczynski) greater than around 0.9. Basically, all measures of association including the recommended Bray & Curtis (Czekanowski) and Gower Metric *underestimate* the association between objects when they do not have sufficient overlap. TRNA may be able to re-estimate these larger association values by a shortest path (network). In this case, the result is likely to be an over-estimation of true association between distant pairs of objects. This may be preferable to *tied* association values of '1.0'. The alternative is to rely on the clustering or ordination phase to get around the problem. No guarantees.

My approach is always to look at the *histogram* of association measures. You can use ASON option 12 to do this. It will hopefully give you some indication of the structure in your data as well as the limitations of the measure of association used. A discontinuity around 0.9 is not unusual for data where many objects have limited or no overlap. If real discontinuities in the data are obvious here, eliminating the outliers is probably a good idea; once identified, they contain little further information. Use MASK to knock out the offending objects.

Classification

If a hierarchical classification is required and the number of objects is less than 100, use:

FUSE (defaults)
DEND
GDEF (look at dendrogram to guess number of groups)

FUSE will optimise the hierarchy and not the groups that you may subsequently derive. If you wish to optimise the groups *or* you have more than 100 objects, a superior approach is to use non-hierarchical clustering through:

ALOC

As can be seen, ALOC is equivalent to running FUSE, DEND and GDEF. You may obtain a dendrogram of the *groups* by copying the inter-group association file -*.sag* to -*.ASO*, and altering the number of objects via PRAM to be the number of groups.

Ordination

Some type of ordination technique should always be used with any Pattern Analysis. If there are more than 200-500 objects (depending on the system **PATN** is running on), ordinate the groups by way of an inter-group association matrix rather than an inter-object one.

If you have used the Bray & Curtis (Czekanowski) or Kulczynski association measures, then a *hybrid* type of ordination would be appropriate. To achieve this use the default options in the SSH module. SSH has been designed to be robust when the nature of the variation of the attributes is basically unimodal as against linear. The problem arises once two objects fail to have any overlap in terms of attributes. When this occurs, association values greater than 0.9 are under estimated. *Hybrid* type SSH, treats associations below the threshold as being *ratio* accurate while those above are considered only *ordinally* accurate.

There is good reason to consider the approach as robust across a wide range of pattern analysis problems. Consequently SSH should be used in preference to principal components, principal co-ordinates, reciprocal averaging/correspondence analysis or other scaling techniques.

Networks

There are only few options here. Use NNB followed by MST and BOND. Taken together, they should provide a reasonable network view of the data. If the dataset is large, then it may be better to use the groups in this step rather than the objects.

Post-Processing

Use GSTA. Select the master option according to the data type (ordinal-ratio or presence/ absence). Both options will have to be run if a set of group centroids is required for ordinal-ratio data. GSTA should provide a good introduction to the contributions of attributes to your classification. Remember, that GSTA doesn't require the intrinsic data (the data used in the classification), it can use **any** data so long as the number of objects matches in **number** and **sequence**. For example, a classification based on say hydrologic attributes can be evaluated on topographic attributes. Great fun.

To display ordination results use SCAT or TSPN (if you have purchased ACROSPIN for US$27). SCAT provides a variety of methods for assisting the interpretation of results. Sequence numbers should be the annotation type for the first display (the number of displays depends on the dimensionality). In addition, a useful/ powerful technique is to use the 'z-value' option in SCAT to plot the value of your original attributes (if any) on the ordination x-y base. The module PCC provides a neat numeric alternative to this, but using your eyes with SCAT is less fallible.

If you would like to use PCC, the output file is in standard **ordination** format (-*.pcc*). To plot the results, reset the number of rows using PRAM to the number of rows PLUS the number of columns. When SCAT asks how many columns there are in the ordination file (-*.pcc*) use one plus the actual number. The last column (for the attributes only), contains the correlation coefficient. Use the 'z' option to plot the correlation. This method provides a useful display.

For a more comprehensive integration of classification, ordination, and networks use TSPN as a pre-processor to the SPIN (tm) program.

If a comparison between a number of classifications on the same set of objects is required, my suggestion is to use TRNA to subtract the appropriate association matrices and classify the result! Pairs of values that are similar will be classified together and vice versa. An alternative is to use RIND. This procedure uses a modified Rand index to compare two partitions of the same set of objects. The number of partitions in each classification does not have to be the same.

Another alternative evaluation procedure for a set of groups from a classification is ASIM; an implementation of Clark and Green's ANOSIM algorithm. This procedure evaluates random re-allocation of objects between groups on the basis of association values.

If comparing ordinations, use PROC; Procrustes rotation. This does an A -> B fit where A and B are two ordination files. Procrustes permits A to be scaled so as to best fit B. Fit measures and stresses for each object are produced.

Once a reasonably complete analysis has been performed, the data should present few secrets. It is appropriate to perform the initial analysis on the complete attribute set and then refine the number and weighting of attributes in conjunction with the association measure. Subsequent pattern analysis adds further refinement and statistical methods may be used to provide confirmation of trends. Remember that **PATN** contains a very wide range of tools that can be used in a variety of ways. While some pragmatism is required in some areas (especially the choice of association measure), 'mixing and matching' modules can provide an almost infinite number of pathways. Suggestions on improvements in the use of **PATN** or any algorithms is always welcomed.

ERRORS

"To err is human; to really foul things up you need a computer!"

A corollary to Murphy's Law states: 'It is impossible to make anything foolproof because fools are so ingenious'. Every attempt has been made to make **PATN** as error-free as possible, however, considering the cunning of users, not to mention the complexity of the package, errors are possible at the most unwanted times. Sorry! With the number of options and sub-options there is just no way to test all possible pathways in **PATN** this millennium!

If you don't understand the error, don't panic - there is a better than even chance that it can be corrected with minor surgery. Experience suggests that most errors are due to the absence of required files or incorrect parameters. These should be reasonably obvious. If you don't have a FORTRAN error and can't understand what's going on, the procedure to follow is:

1. Check that all environmental parameters in PATN.PRM are correct and match your data accurately.

2. Make sure that you're using the correct data file and that it's contents are correct (use DATN option 2).

3. If all else fails: read the documentation, and in desperation, haggle with the author:

Blatant Fabrications
43 Harpers Road
Bonnet Hill, Tasmania
Australia 7053

Phone: +61 3 6229 1910

INDEX